UNIVERSITY OF CALIFORNIA

Los Angeles

# Analog and Mixed-Signal Circuit Analysis and Design: A Data-Driven Approach

A dissertation submitted in partial satisfaction

of the requirements for the degree

Doctor of Philosophy in Electrical and Computer Engineering

by

Zhuofu Tao

2025

© Copyright by
Zhuofu Tao
2025

ABSTRACT OF THE DISSERTATION

Analog and Mixed-Signal Circuit Analysis and Design:
A Data-Driven Approach

by

Zhuofu Tao
Doctor of Philosophy in Electrical and Computer Engineering
University of California, Los Angeles, 2025
Professor Lei He, Chair

Analog/mixed-signal (AMS) circuits traditionally require manual design efforts by experienced engineers. This process is labor-intensive, expensive, and often lacks explainability. Recently, researchers have employed large language models (LLMs) in various fields, showing potential for automating AMS circuit design as well. However, while state-of-the-art LLMs possess significant knowledge, they are incapable of managing the entire design process alone. To address this challenge, we present a series of data-driven methods to enhance AMS circuit understanding and design automation.

We begin by addressing the multimodal LLM challenge of understanding schematics and parsing circuit topology. This limitation currently hampers the use of abundant schematics available in academic literature. We propose a schematic parsing algorithm based on advanced computer vision techniques to convert schematics directly into netlists. Initially, we train an object detection model based on YOLO to identify individual schematic elements. Subsequently, we employ an instance segmentation model based on U-net to detect wires and collect connectivity between elements to generate netlists. By processing 2,686 schematics

from literature with this data transformation pipeline, we create the AMSnet dataset. To enrich its data modality, we manually annotate the schematics with functional traits and reconstruct them in OpenAccess format, making them usable in popular EDA software.

With the data support from AMSnet, we are ready to automate circuit design. We introduce AMSgen, an LLM-driven design flow that spans from performance specifications to post-layout simulation. This process begins with a series of LLM inquiries to derive a block-level design strategy, which we convert into a list of relation-label pairs. Using these pairs, we query our dataset to retrieve subcircuit blocks and assemble them into a complete circuit topology. Next, we utilize Bayesian optimization (BO) to determine circuit sizing. This algorithm intelligently selects sets of parameters and performs simulations to ascertain the corresponding performance. Through a series of trials and errors, the algorithm eventually converges on an optimal set of parameters that achieve desired performance. If performance goals remain unmet after a maximum number of trials, we revisit the topology generation step, using the actual performance as a reference to suggest alternative circuit topologies through LLM knowledge. Our method is evaluated in two case studies, where we successfully generate an operational amplifier and a comparator without human intervention. Additionally, with support from analog cells (Acell), we introduce an automated layout step within the sizing loop, ensuring that the generated circuits consider physical design aspects such as parasitic effects, thereby enhancing reliability.

A limitation of AMSgen is its reliance on BO for end-to-end simulation, which can be costly for larger circuits with more sizing parameters, as the search space grows exponentially. To extend our automated design process to larger circuits, we leverage analytical equations and propose a hierarchical design flow. Inspired by traditional engineering approaches, we first derive performance requirements for the subcircuits from the overall specifications, then optimize them independently. This significantly reduces the number of parameters we need to optimize in each loop, making the design approach more feasible. The entire process is driven by a dependency graph, ensuring that all theoretical calculations and subcircuit

optimizations are conducted in order, populating each parameter before it is required. This approach is evaluated on a SAR ADC (successive approximation register analog-to-digital converter) design, enabling the production of ADCs with 12-14 bits of resolution and up to 1MHz of sampling frequency, again without human intervention.

In summary, we propose a series of methods to collect, transform, and utilize AMS circuit data in automated AMS circuit design. Our approaches allow us to design complex circuits such as SAR ADCs, comprising over 200 devices, with minimal human effort beyond the data collection step. For future work, our methods could extend to additional features such as testbench generation and further automation in data annotation to reduce human effort significantly.

The dissertation of Zhuofu Tao is approved.

Yang Zhang

Lin Yang

Anthony John Nowatzki

Lei He, Committee Chair

University of California, Los Angeles

2025

*To my familiy*

TABLE OF CONTENTS

**1 Introduction** . . . . . **1**

1.1 Background . . . . . 1

1.2 Approach . . . . . 2

**2 AMSnet: Schematic-Netlist Database** . . . . . **5**

2.1 Introduction . . . . . 5

2.2 Related Work . . . . . 7

2.3 Schematic Parsing Algorithm . . . . . 8

2.3.1 Element Detection . . . . . 9

2.3.2 Connectivity Detection: Traditional Approach . . . . . 11

2.3.3 Connectivity Detection: Machine Learning Approach . . . . . 16

2.3.4 Netlist Generation Results . . . . . 19

2.4 AMSnet Database . . . . . 22

2.4.1 Digital Schematic Generation . . . . . 23

2.4.2 Functionality Annotation . . . . . 24

2.5 Discussions and Applications . . . . . 26

**3 AMSgen: LLM-Driven Automated Circuit Generation** . . . . . **29**

3.1 Introduction . . . . . 29

3.2 Related Work . . . . . 30

3.2.1 Machine Learning for AMS Circuit Design . . . . . 30

3.2.2 Sizing . . . . . 31

3.3 AMSgen Workflow . . . 32
3.3.1 Topology Generation . . . 33
3.3.2 Constraint-Augmented Sizing . . . 37
3.3.3 Topology Regeneration . . . 40
3.4 Experiments . . . 42
3.4.1 OPAMP Design . . . 43
3.4.2 Comparator Design . . . 48
3.4.3 Robustness Studies . . . 51
3.5 Discussions . . . 53

**4 Automated SAR ADC Design Using Analytic Equations . . . 55**
4.1 Introduction . . . 55
4.2 Preliminaries . . . 58
4.2.1 SAR ADC Architecture . . . 58
4.2.2 Bayesian Optimization . . . 58
4.3 Algorithms . . . 59
4.3.1 Subcircuit Design Specifications . . . 61
4.3.2 Dependency Graph Construction . . . 65
4.3.3 Optimization Procedure . . . 66
4.4 Experimental Results . . . 67
4.5 Discussion and Future Work . . . 69

**5 Conclusion . . . 71**

**References . . . 83**

LIST OF FIGURES

2.1 LLM correctly answers verbal AMS question . . . . . 5

2.2 MLLM (GPT-4) fails to understand schematic. . . . . 6

2.3 Schematic Parsing Pipeline
(a) Element Detection (b) Connectivity Detection (c) Netlist Generation . . . . 9

2.4 Grouping neighboring elements for net detection . . . . . 12

2.5 Handling intersections for net detection . . . . . 13

2.6 Special transistor symbol with a crossing wire . . . . . 14

2.7 An example of element pin ordering . . . . . 15

2.8 Examples of noisy schematics. (a) Overlaid markings (b) Partial highlighting . . 16

2.9 The proposed two-stage net detection method. (a) Semantic segmentation on the circuit wires to delineate the wires and background areas. (b) Detecting *connected components*, then perform split and merge operations at intersection points to identify the nets. . . . . 17

2.10 Results for element detection (top), net detection (middle), and Spectre format netlist generation (bottom), from the U-net procedure. Columns (a), (b), and (c) present examples from easy, medium, and hard splits respectively. . . . . 20

2.11 (a) An example of element labeling, (b) corresponding annotations. . . . . 26

2.12 (a) 5-Transistor OPAMP, (b) Telescopic cascode OPAMP, (c) Testbench for gain, (d) Testbench for common mode gain, (e) Corresponding knowledge graph diagram. 27

3.1 Proposed AMSnet dataset and AMSgen pipeline flowchart. . . . . 33

3.2 LLM conversation: from performance specification to design strategy (shortened, full version in the Appendix section 5). The green text presents CoT prompt. . 34

3.3 Block-level circuit for 2-stage OPAMP . . . . . 35

3.4 LLM conversation: from circuit characteristics to extract relation-label pairs (shortened, full version in the Appendix section 5) . . . . . 35

3.5 Visualization of the retrieved circuit components followed by the assembled result. The purple arrows illustrate circuit-component-level connectivity. (a) Bias circuitry, (b) First stage, (c) Miller compensation, (d) second stage, (e) Assembled 2-stage OPAMP, symmetric transistors identified from local annotations are highlighted in the same color. . . . . 44

3.6 Bayesian optimization trajectory . . . . . 46

3.7 Visualization of the retrieved circuit components followed by the assembled result. The purple arrows illustrate circuit component -level connectivity. (a) Bias circuitry, (b) First stage, (c) Miller compensation, (d) second stage, (e) Regenerated 2-stage OPAMP. Symmetric transistors are highlighted in the same color. . . . . 47

3.8 Bayesian optimization trajectory for the regenerated design of higher DM gain . 48

3.9 Visualization of the retrieved strong-arm latch comparator. Symmetric transistors are highlighted in the same color. . . . . 49

3.10 Bayesian optimization trajectory for strong-arm latch comparator . . . . . 50

3.11 Topology generation replayed 10 times . . . . . 52

3.12 Bayesian optimization replayed 10 times . . . . . 52

4.1 Automated Design Framework Overview . . . . . 56

4.2 A typical SAR ADC architecture, which is assumed in this work . . . . . 59

4.3 Circuit topologies of SAR ADC subcircuits . . . . . 60

4.4 Comparator propagation delay definition: $t_2 - t_1$ . . . . . 63

4.5 Timing diagram of the clock signals . . . . . 64

4.6 The flowchart of the proposed automated SAR ADC design methodology . . . . 65

4.7 Dependency graph example, variables and equations are explained in detail in Fig. 4.6 . . . . . 66

5.1 LLM prompt: from performance specification to initial design strategy . . . . . 73

5.2 LLM response: from performance specification to initial design strategy . . . . . 74

5.3 LLM prompt: from initial design strategy to relation pairs . . . . . 75

5.4 LLM response: from initial design strategy to relation pairs . . . . . 75

5.5 LLM prompt: from performance specification to revised design strategy . . . . . 76

5.6 LLM response: from performance specification to revised design strategy . . . . 77

5.7 LLM prompt: from revised design strategy to relation pairs . . . . . 78

5.8 LLM response: from revised design strategy to relation pairs . . . . . 78

5.9 LLM prompt: from performance specification to design strategy . . . . . 79

5.10 LLM response: from performance specification to design strategy . . . . . 80

5.11 LLM prompt: from design strategy to relation pairs . . . . . 81

5.12 LLM response: from design strategy to relation pairs . . . . . 81

5.13 LLM prompt and response: tie breaker between valid topologies, continuing on the context of Fig. 5.10 . . . . . 82

LIST OF TABLES

2.1 Netlist generation F1 scores . . . . . 21

3.1 Designs optimized by Bayesian optimization . . . . . 46

3.2 Experiment cost for case studies . . . . . 47

3.3 Designs optimized by Bayesian optimization . . . . . 51

4.1 Variables and Relations . . . . . 60

4.2 Comparison of different works . . . . . 67

4.3 Runtime Breakdown in seconds (BO iterations), number of parameter is listed next to subcircuit name. . . . . 68

# ACKNOWLEDGMENTS

I would like to express my deepest gratitude to my advisor, Professor Lei He, for his invaluable mentorship throughout my Ph.D. journey and for showing me how to conduct meaningful research. I am also thankful to my committee members, Professors Tony Nowatzki, Lin F. Yang, and Yang Zhang for their guidance. Special thanks to Professor Zhiping Yu, whose experience and advice helped shape my progress. I am especially grateful to Professor Ting-Jung Lin and Chen Wu for their generous assistance along the way. I was fortunate to collaborate with Yichen Shi, whose knowledge and insights enriched my work. I would also like to thank Zhongyi Li for the dedication and reliability throughout our research. Finally, I thank my parents and friends for their unwavering support and encouragement throughout this journey.

# VITA

2015–2020 B.S. (Computer Science), University of Waterloo

2020–2025 M.S. and Ph.D. (Electrical and Computer Engineering), UCLA

# PUBLICATIONS

- **Z. Tao**, Y. Shi, Y. Huo, R. Ye, Z. Li, L. Huang, C. Wu, N. Bai, Z. Yu, T.-J. Lin, and L. He, "AMSnet: Netlist Dataset for AMS Circuits," in 2024 IEEE LLM Aided Design Workshop (LAD), pp. 1–5, IEEE, 2024.

- Y. Shi, **Z. Tao**, Y. Gao, T. Zhou, C. Chang, Y. Wang, B. Chen, G. Zhang, A. Liu, Z. Yu, T.-J. Lin, and L. He, "AMSnet-KG: A Netlist Dataset for LLM-based AMS Circuit Auto-Design Using Knowledge Graph RAG" in ACM Transactions on Design Automation of Electronic Systems, 2025.

- Z. Li, **Z. Tao**, Y. Zhou, Y. Shi, Z. Yu, T.-J. Lin, and L. He, "Automated SAR ADC Sizing Using Analytical Equations," in 2025 IEEE International Symposium of Electronics Design Automation (ISEDA), IEEE, 2025.

- Y. Shi, **Z. Tao**, Y. Gao, L. Huang, H. Wang, Z. Yu, T.-J. Lin, and L. He, "AMSnet 2.0: A Larger AMS Database with AI Segmentation for Net Detection," in 2025 IEEE LLM Aided Design Workshop (LAD), IEEE, 2025.

# CHAPTER 1

# Introduction

## 1.1 Background

Analog/mixed-signal (AMS) circuit design remains one of the most intricate and labor-intensive areas of electronic system development. Unlike digital design, which benefits from a high degree of automation and modular abstraction, AMS design continues to rely heavily on human expertise, domain-specific heuristics, and trial-and-error tuning. Crafting even relatively simple AMS blocks requires a deep understanding of both device-level physics and system-level specifications, along with an ability to navigate the subtleties of real-world parasitics, process variations, and layout constraints. Much of the design knowledge is tacit—passed down through experience rather than textbooks—and the iterative process of refining a circuit often hinges on intuition developed through years of hands-on work in lab and simulation environments.

The difficulty in formalizing this knowledge has made AMS design automation a long-standing research challenge. Many steps in the process are not easily captured by deterministic rules or symbolic reasoning. This lack of explainability, combined with the non-convex and environment-dependent nature of AMS design spaces, means that traditional algorithmic approaches struggle to replicate the decisions made by expert designers. While electronic design automation (EDA) tools have advanced in certain aspects of verification and layout, true automation of the design phase—especially for analog and mixed-signal circuits—remains largely unrealized.

Recently, the emergence of large language models (LLMs) has reignited interest in automating complex, expert-driven workflows. LLMs have demonstrated remarkable generalization across diverse tasks, with some even exhibiting behavior akin to human reasoning and problem-solving. Their ability to encode and retrieve vast amounts of knowledge opens the door to a new paradigm of design automation—one that could potentially bridge the gap between rigorous theoretical analysis and the nuanced, experiential knowledge used by human designers. In principle, LLMs could learn to emulate expert behaviors, respond to design constraints, and adapt to iterative feedback in ways that go beyond conventional rule-based systems.

However, when applied to AMS circuit understanding and generation, current LLMs show significant limitations. They struggle with tasks such as interpreting schematics, generating syntactically and electrically correct netlists, and producing reasonable component sizing. These frequent errors suggest that state-of-the-art LLMs lack sufficient domain knowledge in AMS electronics. Given that LLMs are fundamentally data-driven models, their performance can be greatly improved by curating and supplying high-quality domain-specific data. This motivates a new research direction: developing data-centric pipelines that enhance LLM capabilities in AMS circuit analysis and design, thereby moving closer to the long-standing vision of robust and reliable AMS design automation.

## 1.2 Approach

To support large language models (LLMs) with the domain-specific knowledge they currently lack, we first propose a comprehensive dataset—AMSnet—presented in Chapter 2. AMSnet is designed to mirror the diverse modalities of information that analog/mixed-signal (AMS) engineers routinely engage with across the design process. These modalities include schematic diagrams, netlists, functional annotations, layout metadata, and performance metrics, among others. By encompassing a broad spectrum of design representations, AMSnet

enables a more complete learning signal for both model training and retrieval-augmented reasoning.

To construct AMSnet, we developed a cross-modality data transformation pipeline capable of converting human-readable schematics into machine-interpretable netlists. This pipeline leverages computer vision techniques to detect symbols, terminals, and wires within schematic diagrams, reconstruct their topological relationships, and generate accurate netlists in Cadence Spectre format. Additionally, to facilitate seamless integration with mainstream EDA tools, we created a utility to digitally rebuild schematics in environments such as Cadence Virtuoso. Recognizing that much of AMS expertise is implicit or experiential, we also manually annotated a subset of circuits with design rationales, functional descriptions, symmetry constraints, and performance trade-offs. To ensure scalable and efficient data access, the entire dataset was encoded into a knowledge graph, where circuits and annotations serve as nodes, and design relationships and structural hierarchies define the edges.

Building upon the data infrastructure provided by AMSnet, Chapter 3 explores an automated design framework targeting small-scale AMS circuits, such as operational amplifiers and comparators. While state-of-the-art (multi-modal) LLMs are not yet capable of directly generating error-free AMS netlists, they do exhibit a promising ability to outline high-level design strategies and architectural block diagrams. We exploit this by tasking the LLM with generating abstract design blueprints, then performing knowledge graph retrieval-augmented generation (KG-RAG) to fetch and assemble relevant circuit fragments from AMSnet. Once a circuit topology is established, we apply Bayesian optimization to determine optimal device sizing. Critically, our use of symmetry annotations from AMSnet allows for significant pruning of the search space, improving both speed and quality of convergence. Furthermore, by leveraging abuttable analog cells (Acells), we automate layout generation and use post-layout simulation—rather than idealized pre-layout results—as the feedback loop for optimization, capturing parasitic effects and thus delivering far more realistic performance estimates.

Despite the success of this workflow in small-scale settings, practical limitations emerge when scaling to complex circuits. Bayesian optimization requires repeated interaction with computationally expensive simulators; while this is tractable for circuits with tens of parameters, it becomes intractable for large-scale systems like analog-to-digital converters (ADCs), which may involve thousands of parameters and simulation runtimes of several hours. Chapter 4 addresses this challenge by introducing a hierarchical design flow. Instead of optimizing the full circuit holistically, we decompose it into smaller subcircuits. Using a library of analytical performance equations, we approximate how global performance metrics decompose into local targets for subcircuits. A dependency-graph-based optimization scheduler then orchestrates subcircuit optimization in topological order—ensuring that any performance metric dependent on upstream components is computed only after its inputs are resolved. Using this approach, we successfully designed successive-approximation register (SAR) ADCs with varying specifications, reducing end-to-end design time to around an hour with zero human intervention after specification input—even though a single simulation of the full ADC can take over five hours.

In summary, this dissertation demonstrates that data-driven approaches can play a pivotal role in bridging the long-standing gap between formal engineering knowledge and the experiential intuition required for AMS circuit design. By systematically encoding expert behaviors, structural patterns, and performance annotations into a rich, multi-modal dataset, and integrating this data into intelligent design workflows, we show that it is possible to extend the capabilities of modern machine learning models toward practical and reliable design automation.

# CHAPTER 2

# AMSnet: Schematic-Netlist Database

## 2.1 Introduction

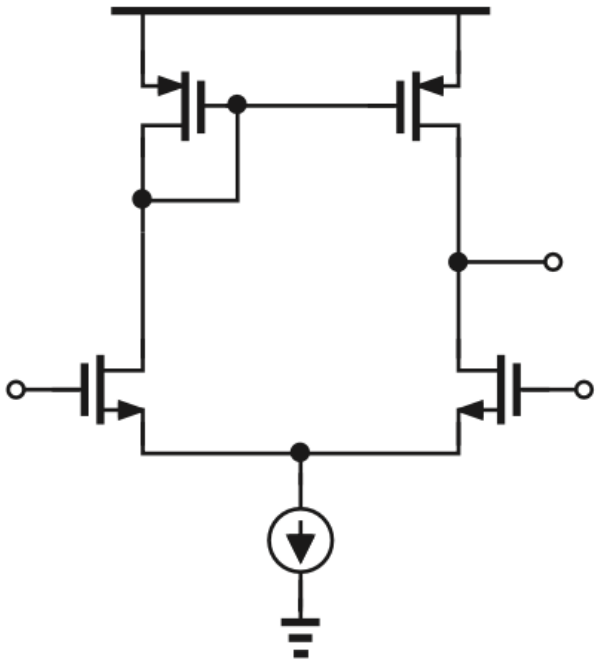
Large Language Models (LLMs) such as GPT-4 have been trained on vast corpora of textual knowledge, demonstrating remarkable capabilities in various domains. These models have shown proficiency in providing reasonable explanations of how analog and mixed-signal circuits function. An illustrative example of this capability is presented below:

> **User:** I have two NMOS transistors which I want to connect into a current mirror, let the two transistors be M1 and M2, provide a list of required connections in this format: "M1.drain - M2.gate"
>
> **AI:** Here is how you can connect them: M1.drain to M2.gate, M1.gate to M2.gate, and M1.source to M2.source. This configuration ensures that both transistors operate in the saturation region, essential for the current mirror to function properly.

Figure 2.1: LLM correctly answers verbal AMS question

As shown in Fig. 2.1, GPT-4 can accurately generate the correct connectivity required to configure a current mirror circuit.

Despite their textual prowess, multimodal LLMs exhibit significant limitations when processing images. Particularly, these models struggle with reading and interpreting circuit schematics accurately. Consider the following interaction:



**User:** How many transistors are there in this schematic?

**AI attempt 1:** In the schematic you've uploaded, there are five transistors visible.

**AI attempt 2:** The schematic you provided contains five transistors.

Figure 2.2: MLLM (GPT-4) fails to understand schematic.

As depicted in Fig. 2.2, GPT-4 fails to accurately count the number of transistors within a given schematic, highlighting its limitations in visual data interpretation. A reasonable hypothesis is that the MLLM recognized the general shape of a 5-transistor operational amplifier (5T OPAMP), and simply responded with the number 5.

The inability of multimodal LLMs to effectively interpret schematics could be attributed to the scarcity of training data. While there is an abundance of programming data available on platforms like GitHub, there is a notable deficiency in data for SPICE or other circuit netlist formats. Consequently, netlists generated by LLMs often cannot be used directly for

simulation or further development, unlike LLM-generated code, which is frequently ready for deployment. Additionally, existing training datasets are predominantly single-modal, lacking sufficient schematic representations adjacent to their corresponding netlists to facilitate effective training.

In response to these challenges, we propose three initiatives:

1. A computer vision method specifically designed to convert schematics into netlists.
2. The application of this method to create AMSnet, a comprehensive database consisting of 2686 pairs of schematics and netlists derived from AMS literature.
3. An annotation format that encapsulates the functionality, strengths, and weaknesses of schematics to enrich the AMSnet database.

The remainder of this chapter is organized as follows. Section 2.2 discusses the recent related works to parse schematics or produce similar databases. Section 2.3 delves into the schematic parsing algorithm. Section 2.4 describes the AMSnet database, including the annotation format and the arrangement of a knowledge graph for efficient retrieval. Section 2.5 discusses potential applications we can build upon AMSnet.

## 2.2 Related Work

Prior to this work, there have been a number of similar AMS datasets. AICircuit [1] provides nine different types of AMS circuits with circuit schematics and a large number of simulation results for sizing. [2] contains five types of AMS circuits, including schematic images, netlists, and testbenches. ALIGN [3] provides a comprehensive collection of AMS circuits, along with well-sized parameters. AncstrGNN Benchmark [4] collects analog circuit netlists without schematic images.

While these datasets have significantly contributed to the development of AMS circuit modeling and optimization, most of them are single-modality—primarily offering netlist data

without corresponding schematic diagrams or detailed annotations. While this suffices for traditional circuit simulation and optimization tasks, such datasets may not be well-suited for training modern multi-modal large language models (MLLMs), which are designed to process and reason over diverse data modalities, particularly images and text. For such models, it is essential to have aligned data tuples that capture the rich cross-modal relationships inherent in circuit design. Therefore, our work emphasizes the need to construct a dataset that not only includes netlists but also provides schematic images, functional annotations (e.g., circuit application, port roles, performance trade-offs), and other descriptive metadata to enable coherent and contextual understanding across modalities.

## 2.3 Schematic Parsing Algorithm

The supervised learning approach for AI training utilizes multimodal data, which involves labeling one data modality with another that we aim to predict or understand. For instance, the renowned computer vision dataset, ImageNet, categorizes images with their respective classifications. This methodology not only laid the groundwork for numerous advancements in computer vision models but also highlights the potential of using similar strategies in other domains. Inspired by ImageNet's success, we propose labeling schematics with corresponding netlists to create a rich dataset that future models can leverage for training.

However, manually reading schematics and labeling netlists is a labor-intensive and costly task, often susceptible to errors. Furthermore, widely used netlist formats like SPICE (Simulation Program with Integrated Circuit Emphasis) only capture the circuit topology, which is inherently non-unique. Modifications such as rearranging lines or renaming devices and nets can be done without altering the circuit's functionality. Given these challenges, it is significantly more advantageous to employ an automated process for parsing schematics and generating corresponding netlists.

Building upon these foundational insights, our proposed schematic parsing algorithm is

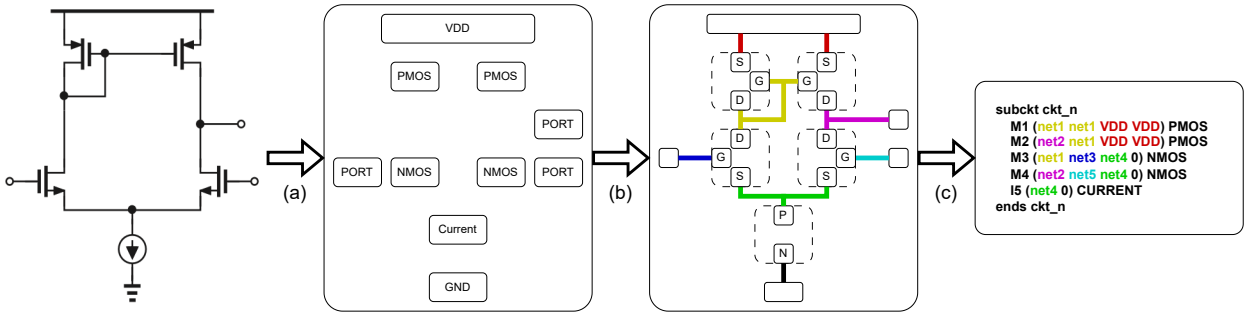



Figure 2.3: Schematic Parsing Pipeline

(a) Element Detection (b) Connectivity Detection (c) Netlist Generation

structured into two sequential steps: element detection and connectivity detection. Initially, the algorithm focuses on element detection, where it identifies and classifies all symbols present in the schematics. These symbols encompass a wide range of **schematic elements**, including basic components like resistors and capacitors, devices such as transistors and amplifiers, and other critical symbols like VDD, ground, junctions, and ports. This step mimics the initial stage of human schematic reading, where one recognizes and categorizes the various elements visually before understanding how they are interconnected. Following this, the connectivity detection phase begins, which involves determining how these identified elements are interconnected within the schematic. This process is akin to tracing the wiring in a physical circuit, establishing a map of how signals flow from one element to another, thus completing the automated interpretation of the schematic layout into a structured netlist. Fig. 2.3 presents an overview of this algorithm.

### 2.3.1 Element Detection

Object detection from images is a fundamental task in the field of computer vision (CV). Various models, such as YOLO (You Only Look Once) [5], have been developed to identify a wide range of objects in images, from inanimate objects like rocks to complex entities such as people. These models are not only versatile in recognizing everyday objects but are also

adept at detecting elements in schematics. They can categorize these elements based on their types and orientations, which is crucial for the subsequent connectivity detection step.

In an ideal world, a dataset would already exist where schematic elements are meticulously labeled with precise *bounding boxes* and *categories*. Unfortunately, as AMSnet might be one of the first, if not the first, schematic-netlist datasets, there is a notable absence of such labeled data. This scarcity compels us to undertake the labeling process ourselves. However, manual labeling, as previously mentioned, is both labor-intensive and prone to errors. To overcome this challenge, we initiate the process by bootstrapping an initial dataset using **template matching**. We then progressively enrich this dataset through semi-supervised learning techniques, which allow for more efficient and accurate label generation as the model improves with incremental training.

Template matching is a conventional algorithm that operates without machine learning. It involves matching a predefined template image to various regions within a source image using a sliding window approach, identifying areas that visually resemble the template. This method is particularly effective in fields like AMS where schematics are typically printed in a uniform style. Much like how every character or digit in a specific font appears identical, schematic elements are also consistently styled across various documents. This uniformity provides an ideal scenario for employing template matching, enabling efficient and accurate detection of these elements in schematic diagrams.

Specifically, at each position of the sliding window, the algorithm calculates the similarity between the template and the *feature*. Whenever the similarity exceeds a certain threshold, the feature is considered a match. In our application, the template may be a manually cropped PMOS transistor in a specific orientation. The matched features would be all PMOS transistors in the same orientation across each schematic. Following [6], the normalized cross-correlation function is presented in Eq. 2.1. $u, v$ are the top-left coordinates of the sliding window, $x, y$ are the dimensions of the template, $f(x, y)$ is a specific coordinate on the feature, $t(x-u, y-v)$ is the matching coordinate on the template, $\overline{f}_{u,v}$ is the mean of

the sliding window, $\bar{t}$ is the mean of the template, and $\gamma(u, v)$ is the similarity between the template and the feature at the current window position. This method significantly reduces our hand-labeling effort from *thousands of elements* across the literature to *tens of unique element types*.

$$\gamma(u,v) = \frac{\sum_{x,y}[f(x,y) - \overline{f}_{u,v}][t(x-u, y-v) - \bar{t}]}{\{\sum_{x,y}[f(x,y) - \overline{f}_{u,v}]^2[t(x-u, y-v) - \bar{t}]^2\}^{0.5}} \tag{2.1}$$

Despite being more cost-effective than manual labeling, template matching is time-consuming compared to deep learning models because each template must be matched individually. In contrast, deep learning models like YOLO can identify all objects simultaneously. For example, applying template matching to about 1000 schematics extracted from [7] using 50 templates takes over an hour, while YOLO requires about 12 minutes for training and only a few seconds for inference. Consequently, we adopt a semi-supervised approach where we initially train a model using data from template matching and then apply this model to label the remaining schematics. After each round of object detection inference, we manually inspect the results. If some elements are not detected, we mark them as new templates and repeat the process. This iterative method ensures that all elements are accurately labeled from the current literature before proceeding to the next, optimizing both accuracy and efficiency. At the end of this process, we manually evaluate the *recall* metric of the element detection results, and the overall recall was 97%. At this point, we have the location, orientation, and class labels for all elements in our schematics.

### 2.3.2 Connectivity Detection: Traditional Approach

With all schematic elements identified, we can now focus on interpreting wires and parsing their connectivity. We propose two different methods for this purpose: a traditional method that utilizes search algorithms and a machine learning method that employs instance segmentation models. We will compare these methods in terms of their strengths and weaknesses.

The traditional method relies on two assumptions: 1) all nets are represented by solid lines on the schematic diagram, and 2) without a junction, two intersecting wires are not considered connected. While these assumptions may not always hold true, they apply to the majority of the schematics we have collected from literature and serve the purpose well.

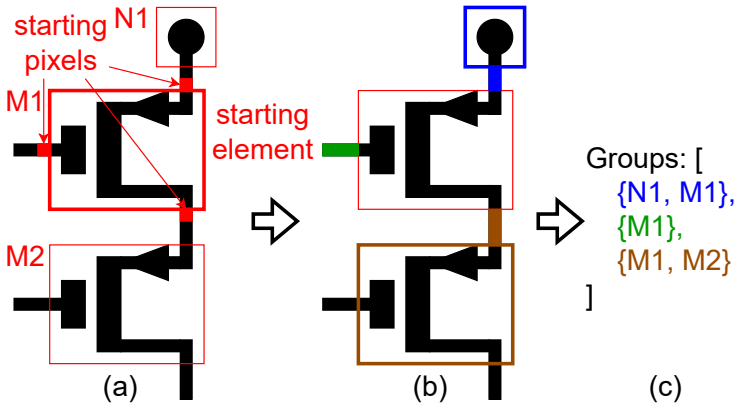



Figure 2.4: Grouping neighboring elements for net detection

The first step involves identifying all immediate connections by breadth-first searching across wire (i.e., black) pixels. Starting from each pixel on each bounding box, the algorithm expands into neighboring wire pixels in all directions until it encounters other bounding boxes, collecting all visited bounding boxes along the way. This step groups elements into clusters, each representing a connection. Fig. 2.4 provides an example. Starting from the wire pixels on the bounding box for PMOS M1, three separate searches result in three groups: N1, M1, M1, and M1, M2. These correspond to the nets connected to the source, gate, and drain pins of M1, respectively. Naturally, we apply this to every element on the schematic.

Each group of bounding boxes falls into one of four possible cases: 1) The group contains only the starting bounding box, as depicted by the green group in Fig. 2.4. In this scenario, no connectivity is detected. 2) The group contains exactly two bounding boxes, illustrated by the blue and brown groups in Fig. 2.4; here, the two bounding boxes are connected. 3) The group consists of an odd number of bounding boxes (more than two), which likely indicates that a junction has been omitted. However, the algorithm cannot determine which

subset of the group is actually connected and which is merely intersecting. Therefore, we flag the entire schematic as an exception; manual attention is required to correct it before further analysis can proceed.

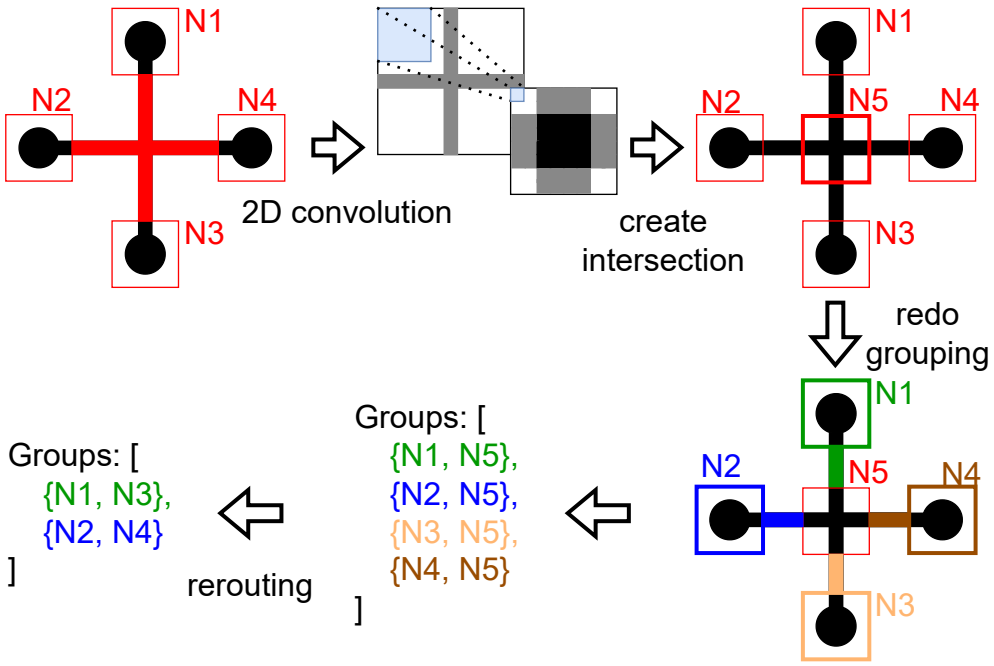



Figure 2.5: Handling intersections for net detection

For the last case, there is an even number (more than two) of bounding boxes in the group, as illustrated in Fig. 2.5. We assume that the schematic has not omitted any junctions; thus, the intersecting wires are not considered connected. To address this, we locate the intersections by applying a 2D convolution to the searched wiring. The area around an intersection typically contains a higher density of wire pixels, allowing us to identify the indices with the highest values as the intersection points, which we then add to our labeled bounding boxes. Subsequently, the algorithm resolves the four-box cluster by repeating the grouping process, connecting each of the four bounding boxes to the intersection. For groups with more than four bounding boxes, each iteration reduces the group size by two until each is left with just two bounding boxes. It is important to note that line weights and layout may influence this step, making the dimension of the convolution kernel a tunable parameter. In

this study, the convolution kernel dimension is set to the average size of the junction points previously identified during object detection. This is based on the assumption that wire intersections generally match the size of junctions, regardless of their connectivity status. Thus, as Fig. 2.5 demonstrates, intersection N5 is roughly the same size as junctions N1-4.

Now that we have identified the intersection point, we order the connections to it by angle, similar to the method shown in Fig. 2.7 but applied to the intersection rather than elements. Given that there are always an even number of connections to each intersection (typically exactly four in practice), we assume that opposite wires are connected to each other. We thus split the list of connections into two halves and pair connections that hold the same position in their respective "half lists." For instance, in Fig. 2.5, the list of connections to intersection N5 sorted by angle could be [N1, N2, N3, N4]. This list would be split into [N1, N2] and [N3, N4], and we would connect N1 with N3, and N2 with N4 to effectively resolve the intersection at N5.

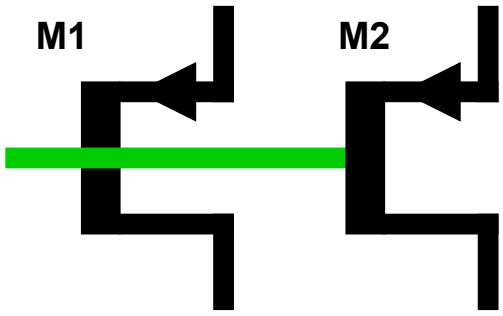



Figure 2.6: Special transistor symbol with a crossing wire

Similar to an intersection, wires are occasionally drawn across transistors, as depicted in Fig. 2.6. Although this setup is similar to an intersection, it requires a slightly different set of rules for resolution. Specifically, in the example shown, the green wire passing through transistor M1 is connected to the gate pins of both M1 and M2. To handle this configuration, we mark the wire segment between M1 and M2 to be ignored during the pin ordering process. Furthermore, we designate M2's gate to "inherit" the net connected to M1's gate. Note that we detect these transistors as a separate class from the ordinary PMOS and NMOS transistors

to support this functionality.

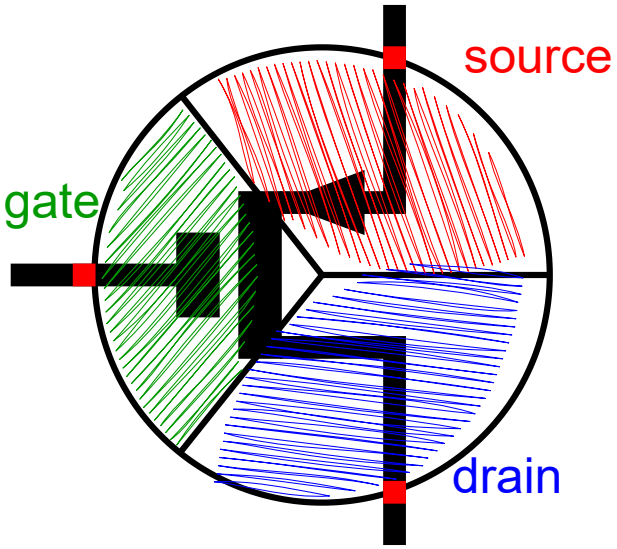



Figure 2.7: An example of element pin ordering

After all the nets are identified, the Spectre netlist format for some element types requires the correct order of connections. For instance, the connections to four-terminal MOSFETs must follow the order of drain, gate, source, and body. Given that the bounding boxes for elements are labeled with orientation, the algorithm can determine a range of angles for each connection, as illustrated in Fig. 2.7. The accuracy of the net labeling process was manually verified, achieving a 96% correctness rate. Any erroneous results are manually corrected to ensure the quality of the data.

With this step completed, the algorithm successfully identifies the circuit elements and their interconnections within the schematic diagrams. Utilizing this information, we are now able to generate the unsized netlist.

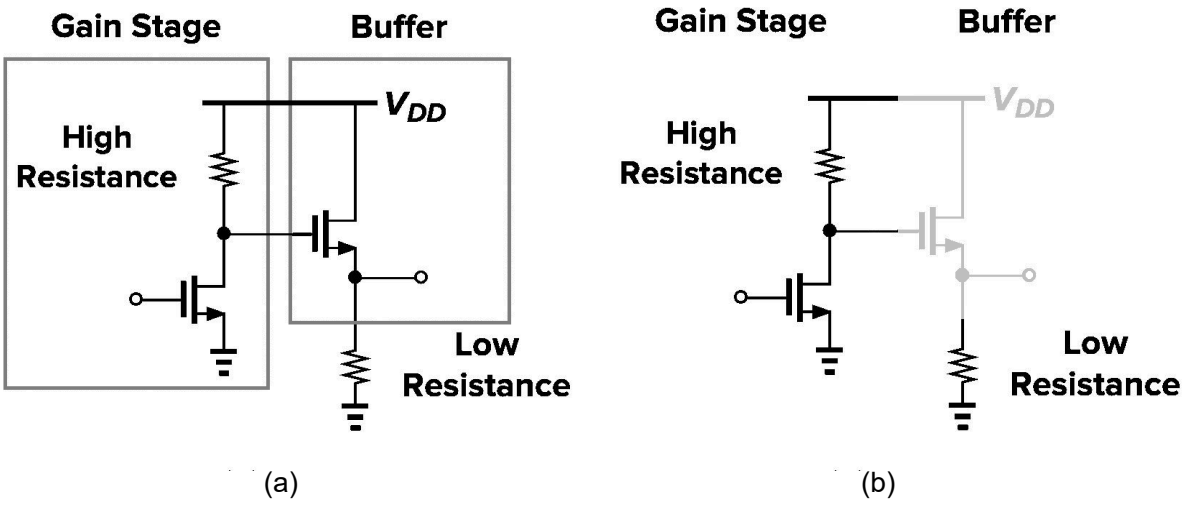



Figure 2.8: Examples of noisy schematics. (a) Overlaid markings (b) Partial highlighting

### 2.3.3 Connectivity Detection: Machine Learning Approach

While the traditional method is effective for many schematics, it does not universally apply to every schematic available, and may fail when its underlying assumptions do not hold. A primary limitation of the traditional method is its reliance solely on observing wire pixels as they appear, without incorporating any *contextual information* such as the location of elements and their connections. Consequently, it might struggle to distinguish actual wires from other illustrative markings (e.g., the boxes in Fig. 2.8(a) could be misinterpreted as wires). To mitigate this, one might assume that such markings typically appear in different colors and employ techniques like binarization to filter them out during a preprocessing step. However, this approach could introduce new problems with different types of illustrations, as shown in Fig. 2.8(b), where parts of the schematic are depicted in a different color and might be unintentionally discarded.

To address these special cases, we propose another method for connectivity detection that employs a deep-learning-based image instance segmentation model. This approach accurately determines whether lines represent wires and identifies which wires are connected to

each other, leveraging contextual information. Instead of hard-coding heuristics to determine connectivity, we rely on a training dataset to guide the decision-making process, and prove to effectively differentiate between wires and other illustrative markings. This approach is significantly more robust than relying on hard-coded heuristics.

Previous research on wire segmentation and net detection often utilizes proposal-based instance segmentation methods, such as YOLO11-seg described in [8, 9]. These methods initially generate potential bounding box proposals, which are then refined through a dedicated segmentation module. However, they frequently struggle to fully encapsulate all pixels of a wire within a single proposal and may inadvertently include pixels from multiple nets, leading to suboptimal segmentation results. Additionally, the survey in [10] explores end-to-end instance segmentation techniques but reports difficulties in accurately distinguishing intersecting nets.

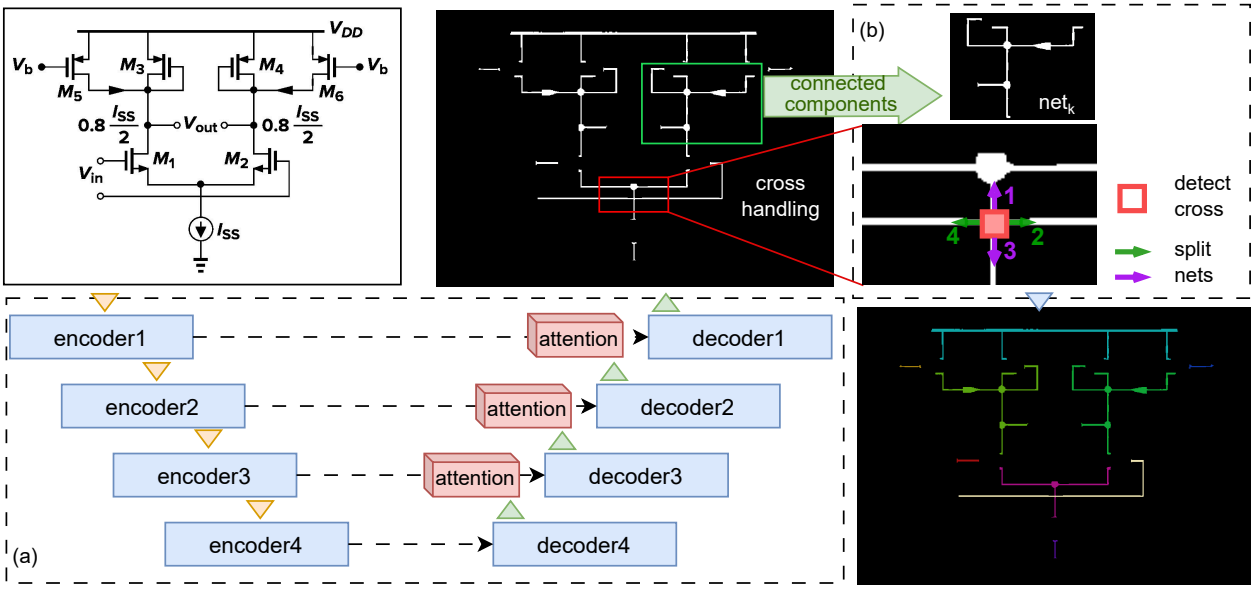



Figure 2.9: The proposed two-stage net detection method. (a) Semantic segmentation on the circuit wires to delineate the wires and background areas. (b) Detecting *connected components*, then perform split and merge operations at intersection points to identify the nets.

To overcome these challenges, we propose a more robust and efficient two-stage approach. This process begins with a semantic segmentation network, specifically U-net [11], to segment all wires and produce wire masks, as depicted in Fig. 2.9(a). These extracted wire masks are subsequently post-processed to accurately separate intersecting nets, as illustrated in Fig. 2.9(b). By eliminating the need for bounding box proposals, our method enhances segmentation accuracy and ensures precise delineation of individual nets, substantially improving the quality of the segmentation outcomes. Note that despite the noticeably inferior performance, YOLO11-seg would not require post-processing steps, therefore completely skips Fig. 2.9(b).

While the previously mentioned methods are effective with *clean* schematics, they often struggle to segment nets accurately in noisy schematics. An example of this is illustrated in Fig. 2.8(a), where rectangles are drawn over the schematics. These *noises*, typically overlay markings used for illustrative purposes, can significantly interfere with the interpretation of schematics.

Data augmentation is a widely used technique in machine learning to enhance the robustness of models by modifying raw training data. In this context, assuming the availability of sufficient training data and a reasonable distribution of data, the proposed model is designed to handle data noise inherently. Therefore, we augment the raw schematics with rectangles and text markings without altering the net mask labeling. We generate *mocked* markings randomly over the schematics, taking care to ensure that these rectangles do not overlap directly with the schematic elements. Following this augmentation step, the trained model is capable of successfully detecting wires through the mocked markings and effectively managing these challenging scenarios.

After device and wire recognition, the flow needs to understand how they are connected to form nets. Given the segmented wire masks from the above steps, those that do not intersect each other belong to separate connected components. We assign a unique net label to each connected component. Fig. 2.9(b) shows an example of wire masks that interconnect and

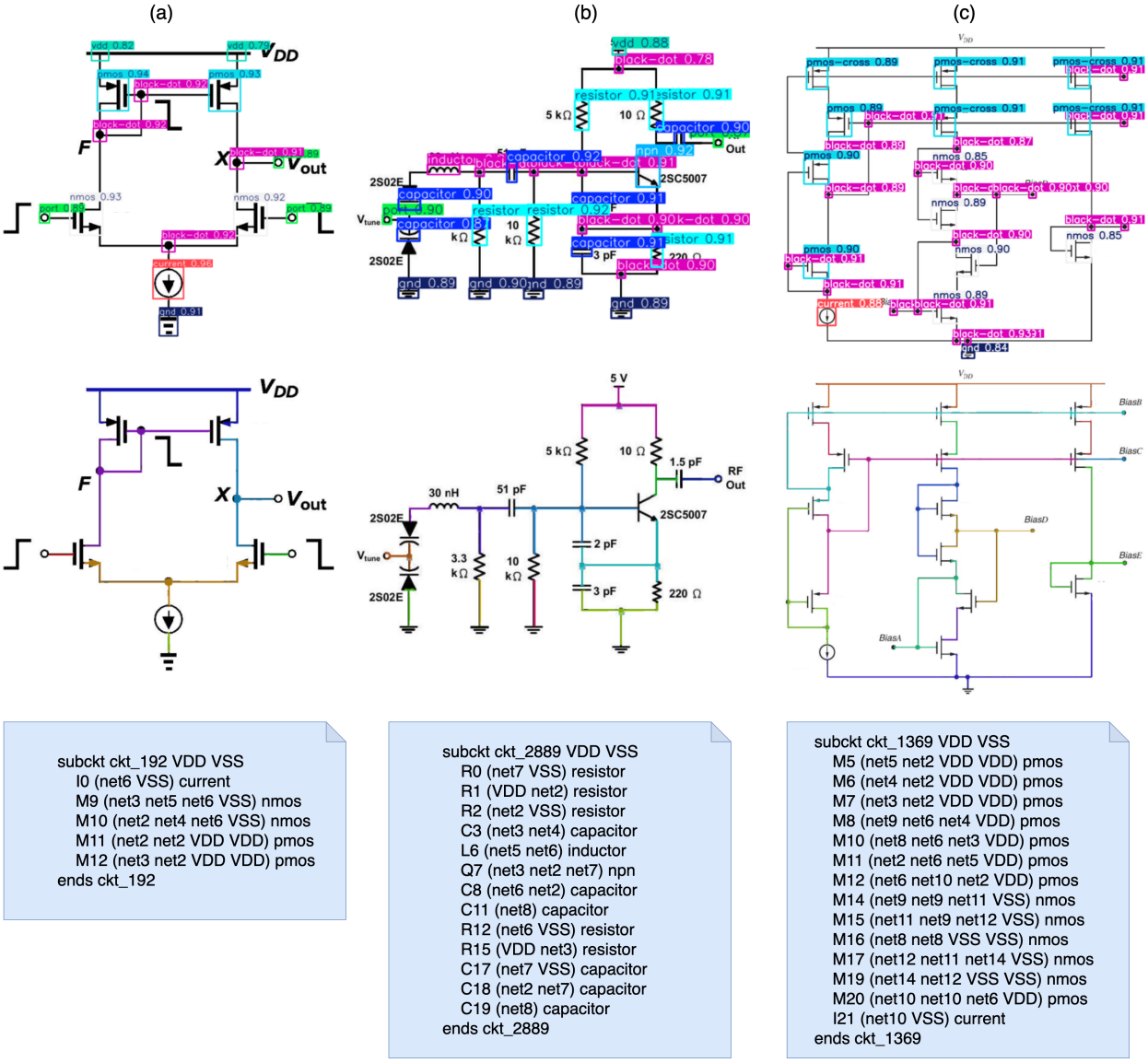
are affected by cross lines. In this case, a mask is applied to the area around the intersection points, which splits the connected component at the intersection. Then, the tool merges the opposite connections to precisely identify the locations of multiple nets. Specifically, it orders all four connections to the intersection based on the angle between the connecting points and the center of the intersection, as shown in the green and purple numbers 1, 2, 3, 4 in Fig. 2.9(b). Finally, the connections 1, 3 and 2, 4 are grouped to resolve the intersection.

At this point, we reproduce the connectivity detection functionality via machine-learning-based methods, and we no longer require context-unaware, hard-coded heuristics and assumptions.

### 2.3.4 Netlist Generation Results

We train our models on a dataset split into training, validation, and testing subsets consisting of 1986, 500, and 200 schematics respectively. For element detection, we employ YOLO11, training it over 500 epochs with a batch size of 16, using the other default parameters of the model. For wire segmentation, we utilize two models: U-net and YOLO11-seg, and we compare their performance. The U-net model is trained for 1000 epochs with a batch size of 32, adhering to the default parameters set for this model. On the other hand, YOLO11-seg is trained for 1500 epochs with a batch size of 16, also using its default settings. All experiments are conducted on an NVIDIA RTX 4090 GPU.

For the evaluation of netlist generation, we categorize three test sets into varying levels of difficulty—easy (80 schematics), medium (70 schematics), and hard (50 schematics)—based on the complexity of the circuit topology as well as the quality and style of the schematics. We manually assess the test set schematics using four metrics: high element count (10 or more elements), crossing wires, overlaid markings, and low resolution. Schematics that meet none of these criteria are classified as "easy"; those that meet exactly one criterion are labeled as "medium"; and those that meet two or more are designated as "hard". Fig. 2.10 presents three examples, each from the easy, medium, and hard splits. Each example includes the




```
subckt ckt_192 VDD VSS
   I0 (net6 VSS) current
   M9 (net3 net5 net6 VSS) nmos
   M10 (net2 net4 net6 VSS) nmos
   M11 (net2 net2 VDD VDD) pmos
   M12 (net3 net2 VDD VDD) pmos
ends ckt_192
```

```
subckt ckt_2889 VDD VSS
   R0 (net7 VSS) resistor
   R1 (VDD net2) resistor
   R2 (net2 VSS) resistor
   C3 (net3 net4) capacitor
   L6 (net5 net6) inductor
   Q7 (net3 net2 net7) npn
   C8 (net6 net2) capacitor
   C11 (net8) capacitor
   R12 (net6 VSS) resistor
   R15 (VDD net3) resistor
   C17 (net7 VSS) capacitor
   C18 (net2 net7) capacitor
   C19 (net8) capacitor
ends ckt_2889
```

```
subckt ckt_1369 VDD VSS
   M5 (net5 net2 VDD VDD) pmos
   M6 (net4 net2 VDD VDD) pmos
   M7 (net3 net2 VDD VDD) pmos
   M8 (net9 net6 net4 VDD) pmos
   M10 (net8 net6 net3 VDD) pmos
   M11 (net2 net6 net5 VDD) pmos
   M12 (net6 net10 net2 VDD) pmos
   M14 (net9 net9 net11 VSS) nmos
   M15 (net11 net9 net12 VSS) nmos
   M16 (net8 net8 VSS VSS) nmos
   M17 (net12 net11 net14 VSS) nmos
   M19 (net14 net12 VSS VSS) nmos
   M20 (net10 net10 net6 VDD) pmos
   I21 (net10 VSS) current
ends ckt_1369
```

Figure 2.10: Results for element detection (top), net detection (middle), and Spectre format netlist generation (bottom), from the U-net procedure. Columns (a), (b), and (c) present examples from easy, medium, and hard splits respectively.

results of element detection, net detection, and the Spectre netlist generated by the U-net-based procedure.

In our experiments, we implement two distinct approaches for wire detection. The first approach utilizes YOLO11-seg [9], which is designed to directly detect and segment wires into separate nets. The second approach involves UNet, which initially detects segments of wire, then uses algorithmic post-processing to separate these segments into distinct nets, aided by crossing points identified during the element detection phase. Accordingly, we report two sets of results, as detailed in Table 2.1.

| Model | Easy | Medium | Hard |
|---|---|---|---|
| Unet [11] | 90.19 | 84.17 | 80.39 |
| YOLO11-seg [9] | 83.62 | 71.10 | 73.11 |

Table 2.1: Netlist generation F1 scores

The evaluation of these results is conducted using a **confusion matrix**. For each netlist component (excluding common elements such as VDD, GND, ports, etc.) and each net connection in the ground truth netlist, the following criteria are applied:

- If a component or connection is correctly predicted in the predicted netlist, it is recorded as a **true positive**.
- If a component or connection is missing from the predicted netlist or is predicted incorrectly, it is recorded as a **false negative**.
- If a component or connection that does not exist in the ground truth is erroneously predicted (e.g., an extra capacitor or an unnecessary connection), it is recorded as a **false positive**.

From these metrics, we calculate precision, recall, and subsequently the **F1 score**, which are reported in Table 2.1. Notably, incorrect type predictions, such as misidentifying a capacitor as a resistor, are also classified as false negatives. This methodology ensures a thorough

and precise evaluation of the effectiveness of our wire detection and netlist generation algorithms.

Given that different netlists can have varying element names and net names, direct comparisons might not accurately reflect the effectiveness of the models. Therefore, we account for all possible permutations between ground truth names and predicted names to ensure a fair evaluation. For instance, consider netlist ckt_192 depicted in Fig. 2.10(a), where the predicted netlist might name elements as I0 and M9-12, and net names as VDD, VSS, and net2-6. If the ground truth labels VDD as "net1", the correct permutation would match net1 with VDD, among other matches. To quantify this, we report the F1 score based on the best possible permutation that aligns the predicted names with the ground truth names. However, as the complexity of computing permutations grows exponentially with the number of elements and nets, we sometimes manually determine the F1 score when the schematics are too complex. Furthermore, since we have the net positions through instance segmentation, we can compare them with the ground truth and secure a subset of net matches, significantly reducing the search space.

## 2.4 AMSnet Database

With the schematic-parsing algorithm at our disposal, we are equipped to build a comprehensive database by collecting and processing raw schematic images. Beyond just the fundamental content of schematics and netlists, we integrate several additional types of information. The schematic parsing process highlights the crucial role of positional information for both elements and wires, which enables us to digitally reconstruct schematic images accurately. Although schematics alone offer limited utility for circuit design automation, their value can be greatly enhanced through functionality labeling.

Section 2.4.1 provides an in-depth explanation of the detailed positional information we extract, utilize, and refine to construct digital schematics. Section 2.4.2 delves into the work

on functional annotation and discusses the *knowledge graph* structure we employ to organize and store data within AMSnet.

### 2.4.1 Digital Schematic Generation

While generating netlists from circuit schematics enables programmatic use and supports downstream tasks, such as the automated parameter sizing method described in the next chapter. Having only the netlist is insufficient for practical adoption. Engineers are far more accustomed to working within EDA environments rather than manipulating raw netlists. Moreover, although our element and connectivity recognition performs reliably, occasional inaccuracies are inevitable. In such cases, visual inspection and manual correction through a schematic view is significantly more intuitive than examining a text-based netlist.

To bridge this usability gap, we introduce a digital reconstruction workflow that recreates the original schematic in the OpenAccess format, compatible with widely-used EDA tools such as Cadence Virtuoso. This enables both manual verification and further design development by engineers in a familiar interface.

Unlike netlist generation, which only requires topological information, schematic reconstruction demands spatial data—specifically, the positions and orientations of elements and the routing paths of interconnects. This process resembles a placement-and-routing step, aiming to preserve the visual structure of the original schematic as faithfully as possible. Our element recognition algorithm already provides the position and orientation of each component, so the main challenge lies in reconstructing the wiring.

To achieve this, we define and extract a set of key points from the wire pixels. These key points fall into three categories:

1. End points: terminal points of wires that connect to components.

2. Diverge points: junctions where multiple wires intersect.

3. Turn points: locations where wires change direction significantly.

We first apply skeletonization to the wire pixels, reducing each interconnect to a one-pixel-wide structure. We then perform a depth-first search (DFS) to identify end and diverge points: a DFS step that terminates defines an end point, while one that splits into multiple traversal branches indicates a diverge point. A second DFS tracks directional continuity along the skeleton. Abrupt changes in direction exceeding a certain threshold are flagged as turn points if they are not already ends or diverges. Finally, we merge all neighboring points.

These extracted key points effectively compress the wire data into a compact representation suitable for reconstruction. The wires are then reassembled in OpenAccess by connecting the corresponding key points.

To improve schematic readability and aesthetics, we apply a post-processing step that aligns adjacent key points. Horizontally or vertically neighboring points are snapped together to straighten wires and produce cleaner, parallel routing. Additionally, pin offsets from each element's location are treated as key points and included in the alignment step to ensure proper visual and functional integration within the schematic.

This reconstructed schematic not only supports usability within EDA software but also serves as a direct visual reference for comparison against the original input. Manual corrections can be applied efficiently, and we envision future enhancements where machine learning models assess the similarity between the original and reconstructed schematics. This could enable automatic alignment and correction, a direction we plan to explore in future work.

### 2.4.2 Functionality Annotation

In addition to schematics and netlists, we also include annotations to enrich the circuit data. We define two types of annotations: *local* and *global*. Local annotations describe one or more elements within a circuit, while global annotations pertain to the entire architecture.

The purpose of local annotations is to guide the *usage* of the circuit. In this work, local annotations are hand-labeled. Examples include identifying nets as inputs, biases, outputs, or labeling groups of elements as building blocks, such as current mirrors or differential pairs. An important type of local annotation is the labeling of sizing constraints. These mark specific sets of elements to adhere to constraints such as symmetry in transistor lengths and widths. These constraints, typically derived from expert insights and design experience, play a crucial role in the sizing process by reducing search space and computation costs. Net labels, on the other hand, are used to connect the pins of circuits to each other and to testbenches.

The purpose of global annotations is to *guide topology selection*, helping to choose one circuit over others. It is common for individual circuits not to have universally recognized names. Instead, they are often described by their elements or known for their specific functionalities. For example, an engineer might describe an op-amp by its stages or choose it for its exceptional performance in a particular area. To facilitate this information retrieval process, we label each circuit with architectural descriptions, such as performance specialties and subcomponents. We take advantage of the fact that the source literature, from which we initially obtain each schematic, likely includes some verbal descriptions within its context. We manually summarize this information into qualitative descriptions in the form of key-value pairs, such as gain: high or load: current mirror.

We take a further step by producing JSON files to introduce more structure into our annotation data. Fig. 2.11(a) illustrates annotations on a circuit, while Fig. 2.11(b) presents the local and global annotation files for the given circuit. Since individual JSON files may share numerous labels, we simplify the data retrieval process and reduce data volume by merging equivalent annotations to form a knowledge graph.

A knowledge graph (KG) is a structured form of knowledge representation that delineates the relationships between entities. In a KG, nodes typically represent entities, while edges denote various semantic relationships between these entities. The fundamental unit

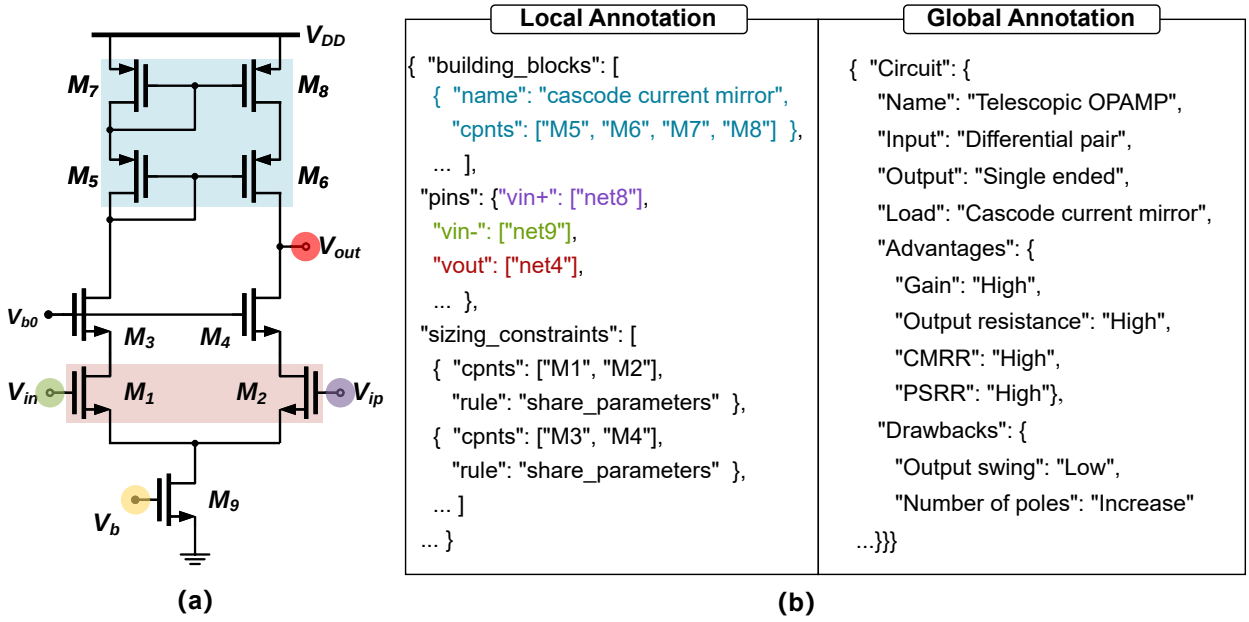



Figure 2.11: (a) An example of element labeling, (b) corresponding annotations.

of composition in a knowledge graph is the "entity-relation-entity" *relation triplet*, where entities are interconnected through relations, forming a graph-like data structure.

For the AMSnet-KG dataset, we compile all schematics, netlists, and annotation data. In this dataset, an entity is defined as either a string or a circuit. The annotation keys and values are represented as simple strings, while the circuit entity encompasses a schematic, a netlist, image attributes such as bounding boxes and net marks, and all associated annotations. Relations serve solely to connect entities and are also represented as strings. Fig. 2.12 illustrates a small-scale KG created from two circuits and two testbenches, showcasing how these elements interlink within the graph.

## 2.5 Discussions and Applications

In this chapter, we introduced AMSnet, a novel multimodal dataset comprising circuit schematics and corresponding netlists. By leveraging a combination of computer vision

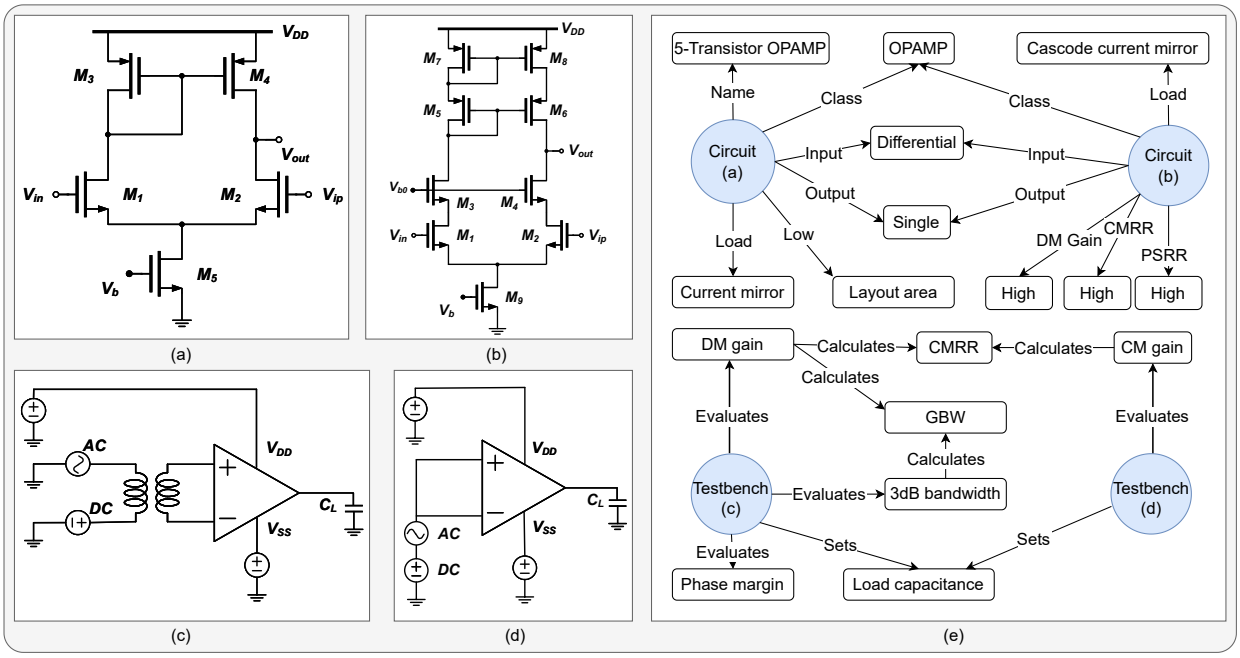



Figure 2.12: (a) 5-Transistor OPAMP, (b) Telescopic cascode OPAMP, (c) Testbench for gain, (d) Testbench for common mode gain, (e) Corresponding knowledge graph diagram.

techniques and traditional algorithms, we successfully transformed complex schematic images into structured netlist representations, laying a solid foundation for advanced AMS circuit analysis and automation.

There are several promising applications of the AMSnet dataset. In the following chapter, we will demonstrate how this database can significantly enhance AMS circuit design automation. Specifically, AMSnet aids in ensuring topological correctness—a critical step, as multimodal large language models (MLLMs) commonly make errors in generating accurate circuit topologies. Moreover, AMSnet provides an ideal environment for developing automated validation tools that verify topology correctness in LLM-generated netlists.

Additionally, the rich annotations within AMSnet facilitate the generation of diverse question-answer pairs suitable for training MLLMs. These pairs cover essential skills in AMS circuit engineering, including circuit perception, analysis, and design reasoning. We intend to explore these training applications extensively in future research.

Looking ahead, we plan to further expand AMSnet to better replicate the practical experiences of AMS engineers. Specifically, future iterations will incorporate detailed simulation data beyond qualitative descriptions, capturing quantitative results, precise waveforms, and interpretations of circuit performance. By incorporating common errors and their corresponding solutions, AMSnet will closely mimic the abundant coding forums currently available to software engineers, significantly elevating the state-of-the-art in AMS design automation.

# CHAPTER 3

# AMSgen: LLM-Driven Automated Circuit Generation

## 3.1 Introduction

Digital circuit synthesis is extensively utilized in electronic design automation (EDA) [12], allowing contemporary large-scale digital integrated circuits (ICs) to advance in accordance with Moore's law. However, the automation in analog and mixed-signal (AMS) circuit design lags significantly behind that of digital design [13]. AMS circuits are primarily custom-designed today, heavily relying on human experts to determine circuit topologies and component sizing. This time-consuming and labor-intensive design process significantly hinders the scalability of AMS circuits.

Large language models (LLMs) have recently shown potential to address various EDA challenges [14], offering new hope for automatic AMS circuit design. However, the availability of high-quality digital circuit data on the internet, such as Verilog code, far exceeds that of AMS circuits, mainly represented by netlists [15]. Consequently, LLMs have been extensively applied in the digital circuit domain, achieving significant advances in RTL code generation [16, 17]. However, their application in AMS circuits remains exploratory. Researchers use LLMs to assist in analog circuit topology design and transistor sizing [15, 18, 19, 20]. Compared to traditional AI methods such as reinforcement learning (RL) and Bayesian optimization (BO), LLMs offer greater flexibility in interaction, knowledge transfer, expansion capabilities, and interpretability of design solutions.

Due to the scarcity of public AMS circuit netlist datasets, LLMs are insufficiently trained

to produce accurate netlists with correct topology and reasonable device sizing [18, 15]. Researchers have enhanced the performance of LLMs in designing AMS circuits by training them with datasets collected for specific circuits [18], or by employing prompt engineering techniques including chain of thought (CoT)[21], ReAct (Reason + Act)[22], and in-context learning (ICL)[23, 24, 15]. Retrieval-augmented generation (RAG) is also used to supplement LLMs with additional knowledge without requiring expensive retraining[20].

Building upon the AMSnet dataset, we propose AMSgen, an automatic AMS circuit generation framework. AMSgen aims to generate fully sized netlists from performance specifications. Starting with high-level design strategies generated using LLM, AMSgen describes the circuit architecture using subcircuits. It autonomously retrieves matching subcircuits from AMSnet and assembles them into a complete topology via predefined block-level schematics. Guided by local annotations to reduce the search space, AMSgen automatically optimizes the transistor sizing through BO. If the generated design does not meet performance specifications, AMSgen adds the current design and achieved performance to ICL as historical unsuccessful designs, and returns to the LLM-based strategy generation step. The process iteratively generates unique new designs until all performance goals are met.

The remainder of this chapter is organized as follows. Section 3.2 discusses recent works related to generating AMS circuit topology, sizing, and layout. Section 3.3 describes the AMSgen workflow. Section 3.4 explores two case studies on OPAMP and comparator design. Section 3.5 discusses the overall contributions from AMSgen and potential future work.

## 3.2 Related Work

### 3.2.1 Machine Learning for AMS Circuit Design

The automatic design of AMS circuits primarily includes topology synthesis and transistor sizing. In the era before LLMs, traditional machine learning or deep learning methods encode AMS circuits into strings [25], trees [26, 27], or graphs [28, 29, 30], and use methods

such as evolutionary algorithms, BO [31, 28], and reinforcement learning [29] for automatic topology design. CktGNN [30] proposes a two-level GNN framework that simultaneously automates circuit topology generation and device sizing. [31] It maps the block-level model of the circuit as a directed acyclic graph (DAG) and uses a variational graph autoencoder and BO for bi-layer optimization.

LLMs demonstrate remarkable capabilities in numerous EDA tasks, such as Verilog and RTL code generation [32]. Recent efforts also explore using LLMs to directly generate analog circuit topology [18, 15, 33, 34]. However, due to the scarcity of AMS circuit topology data in pretrain datasets, these efforts are not very effective. Artisan [18] trains an LLM for the automatic block-level design of OPAMPs. However, the OPAMP-specific knowledge used to train their LLM cannot be smoothly generalized to other types of circuits, such as comparators and LDOs. AnalogCoder [15] does not retrain the model but uses carefully designed prompts to leverage the LLM's capability to generate netlists. Due to the precise requirements of AMS circuits, multiple attempts and iterations are needed to achieve accurate results, if possible. LaMAGIC [33] achieves automated analog circuit design through supervised fine-tuning, automatically designing power converters consisting of three to six components. The transistor-level topological correctness of these methods relies on model knowledge, and may not be robust enough to scale to larger circuits. In this work, the correctness of the generated topology is guaranteed by the correctness of the dataset rather than relying on the LLM response, and is therefore more robust.

### 3.2.2 Sizing

The goal of analog integrated circuit sizing is to determine the parameters of circuit components, such as transistors, within a given topology to meet design objectives. However, the substantial intra-class and inter-class variations, coupled with varying design goals across different circuit types, render the sizing process for analog integrated circuits particularly challenging. Researchers extensively study the automation of this process, primarily cate-

gorizing the methods into two types: 1) knowledge-based and 2) optimization-based. For knowledge-based methods such as [35], circuit designers create predefined schemes and equations to calculate transistor sizes. However, deriving design schemes for every existing circuit topology is very time-consuming and also requires continuous maintenance to keep up with the latest process technologies. On the other hand, LADAC [20] designs the first LLM agent for analog circuit sizing based on ReAct [22]. By constructing an expert knowledge base, integrating ICL and CoT for decision-making, LADAC successfully derives transistor parameters for multiple circuit topologies and satisfies their respective design specifications. Optimization-based methods treat the performance of circuits as a black-box function and use heuristic algorithms [36, 37, 38, 39, 40, 41, 42] or surrogate model approaches for optimization [43, 44, 45, 46, 47]. The use of Gaussian processes as surrogate models in optimization methods is widely studied. ADO-LLM [19] combines an LLM with BO, using both the LLM and the acquisition function to determine the next sampling point. However, it does not discuss how to incorporate PDK technology nodes into their workflow, despite the fact that PDKs play an important role in the sizing process. Previous optimization-based methods include expert insights to reduce search space [48], but do not provide a systematic method to store and use these insights. Therefore, in this work, we propose a universal format to store and use expert insights for transistor sizing.

## 3.3 AMSgen Workflow

The support from AMSnet enables the full automation of AMS circuit design, transitioning from performance specifications to fully sized netlists. This section explores our LLM-assisted, data-driven design pipeline, AMSgen, as illustrated in Fig. 3.1. Section 3.3.1 details the topology generation step. Section 3.3.2 discusses the sizing step, and Section 3.3.3 addresses the topology regeneration step in cases where the initial topology fails to meet performance goals after sizing. The user provides only the performance specifications and

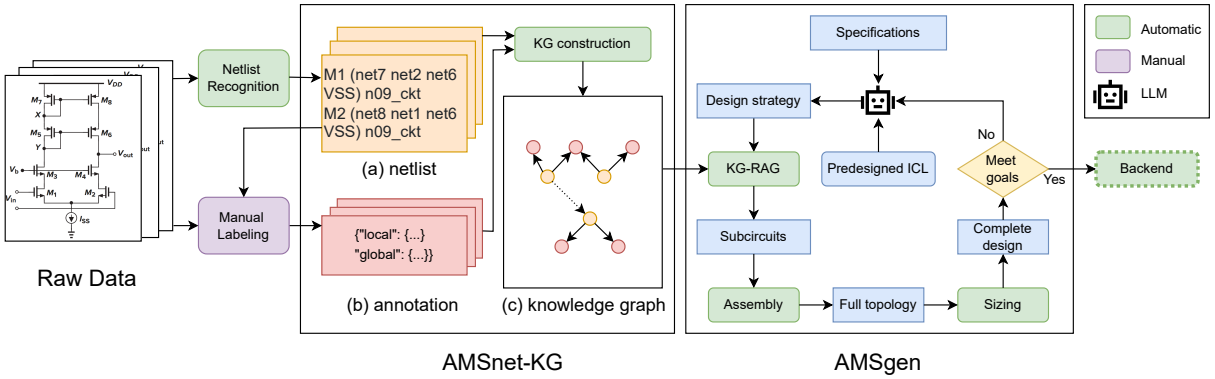



Figure 3.1: Proposed AMSnet dataset and AMSgen pipeline flowchart.

does not interact with the design flow thereafter.

### 3.3.1 Topology Generation

Traditionally, AMS circuits are designed manually by engineers, who begin by selecting circuit topologies and testbenches based on a set of desired performance specifications. Engineers draw on their experience or additional resources to identify circuit topologies that meet the design goals and select testbenches to evaluate each performance metric. However, this process is labor-intensive, time-consuming, and heavily reliant on experience, making it challenging to automate.

With the LLM technology available today, coupled with our AMSnet data support, we can fully automate this process and achieve the same goals. Given a set of performance metrics, AMSgen initially obtains a design strategy from the LLM that outlines the circuit architecture as a collection of subcircuits. It then retrieves corresponding subcircuits and testbenches from AMSnet before assembling them for simulation. Since the subcircuits are pre-validated, this ensures correctness, even when the LLM response is incorrect.

**Design strategy generation:** We design a comprehensive prompt engineering framework, incorporating in-context learning (ICL) [23, 24] and chain of thought (CoT) [21] to generate

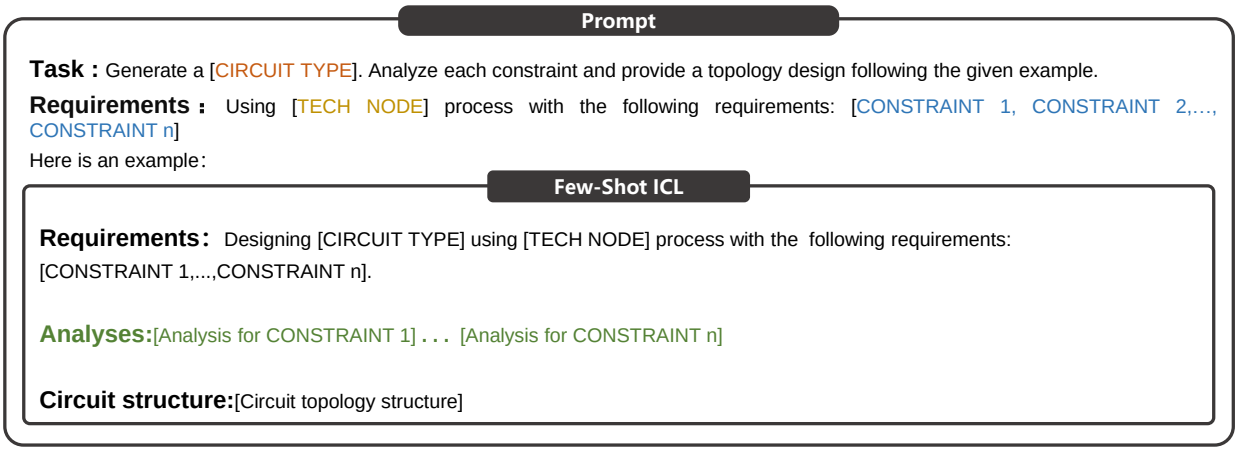



Figure 3.2: LLM conversation: from performance specification to design strategy (shortened, full version in the Appendix section 5). The green text presents CoT prompt.

a circuit topology design strategy. Fig. 3.2 presents an abstract version; full versions are available in the Appendix section 5.

State-of-the-art LLMs possess the knowledge needed to provide design strategies for desired circuits. However, they typically respond in natural text, which may be challenging to extract and use programmatically. To standardize the response format, we manually select an ICL example from a circuit sample set, encouraging the LLM to emulate this output format, which facilitates further application of the response. To prevent the LLM from favoring any specific topology, the ICL examples describe different circuit types to provide formatting guidance only. For instance, we use a comparator design strategy as the few-shot example to generate OPAMPs, and vice versa. We also employ chain of thought (CoT) prompting methods, which have been shown to effectively enhance LLM performance. Specifically, the model is instructed by the example to first qualitatively analyze the requirements in the specifications and explain a thought process before providing the actual circuit design. Although the state-of-the-art LLM can produce reasonable design strategies without ICL or CoT, using these methods significantly improves formatting robustness.

In an ideal scenario, the LLM would be capable of designing the entire circuit topology

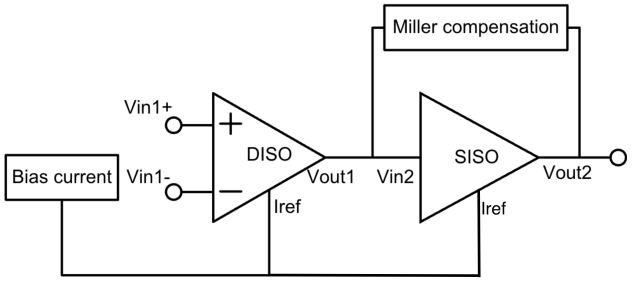



Figure 3.3: Block-level circuit for 2-stage OPAMP

**Prompt: Design Strategy to Relation Pairs**

**Task:** Extract the relationships from the following text into relation pairs for knowledge graph query. Each pair should be in the form of: <RELATION, OBJECT>.

**Note:** RELATION should be performance metrics or building blocks, i.e. gain, power consumption, input, load, etc.
**Note:** OBJECT should be qualitative description for the performance or building block selection, i.e. high, fast, current mirror, latch, etc.
**Example pairs:**
<gain, high>
<offset voltage, low>
<power consumption, low>
<input, differential pair>

**Circuit structure: …**

Figure 3.4: LLM conversation: from circuit characteristics to extract relation-label pairs (shortened, full version in the Appendix section 5)

directly. However, as demonstrated in several existing studies [15, 20], their responses are still prone to errors. To enhance correctness and robustness, we utilize the AMSnet dataset for topology guidance and rely on the LLM primarily for subcircuit selection rather than detailed transistor-level descriptions. Assuming that all topologies stored within AMSnet are correct, the LLM would only need to provide sufficient information to help us narrow down the appropriate circuit or subcircuits to assemble into a complete circuit.

**Circuit Retrieval and Assembly:** Given the target circuit type to create, AMSgen selects a predefined block-level circuit, as illustrated in Fig.3.3, and only needs to fill in the required subcircuits. We do not use the subcircuit level connectivity provided by the LLM; instead, we select subcircuits based on its response. As shown in Fig.3.4, we conduct another LLM query to format the subcircuit descriptions within the design strategy into relations

**inputs:** number of nodes $n$ and edges $m$ in KG, number of design strategy queries $p$;

circuits of desired type $C_{KG} = [c_1, c_2, ..., c_n]$;

relation triplets $R_{KG} = [(c_{r_1}, \text{relation } r_1, \text{label } l_1), (c_{r_2}, r_2, l_2), ..., (c_{r_m}, r_m, l_m)]$;

queries $Q = [(\text{feature } f_1, \text{value } v_1), ..., (\text{feature } f_p, \text{value } v_p)]$;

word Hamming distance scores $S = 0$ for each $C_i$;

**for** $i \leftarrow 0$ **to** $m$ **do**

  **for** $j \leftarrow 0$ **to** $p$ **do**

    $S[c_{r_i}]$ += WordHammingDistance($r_i$, $f_j$) + WordHammingDistance($l_i$, $v_j$);

  **end**

**end**

**return** $C_{KG}$ sorted by $S$ ascending;

**Algorithm 1:** Subcircuit Retrieval and Relevance Ordering

such as ¡name, telescopic¿, ¡load, current mirror¿, ¡gain, high¿, etc., to simulate how engineers recall circuits by their various traits. This process facilitates the retrieval of relevant subcircuits from AMSnet. To standardize the output format, we predesign additional ICL examples and input them into the LLM along with the intended question. Unlike in the design strategy generation, these ICL examples are more impactful; without them, the LLM might provide relation pairs that are syntactically correct but practically useless. For example, without ICL, we often receive pairs such as ¡gain, required¿ rather than ¡gain, high¿. After obtaining the relations, AMSgen searches for the most relevant subcircuits within AMSnet, using Cypher queries for Neo4j. Alg. 1 presents the retrieval and relevance ordering logic.

Specifically, for subcircuit relevance calculation, we use the *word Hamming distance* to compare different terms. For instance, consider the relation triplet (Circuit (b), DM Gain, High), as shown in Fig. 2.12(e) represented by an arrow from *Circuit (b)* to *High* marked as *DM gain*. When matching against the query (Gain, very high), the relation *DM gain*

matches the feature *Gain*. We count the number of word occurrences on both sides, all in lowercase. The relation *dm gain* contains the word *dm* once and *gain* once, while the feature *gain* contains the word *dm* zero times and *gain* once. We compare the two vectors [1,1] and [0,1] to obtain a Hamming distance of 1. Similarly, we compare the label *high* against the value *very high* and obtain a Hamming distance of 1 as well, for a total distance of 2. We rank all matches in ascending order by this distance; the lowest distances have the minimal differences and therefore the closest match. Although this naive matching algorithm works well in our experiments, to enhance robustness, we could replace it with more sophisticated methods, such as language vector similarity or LLM inquiry.

Now that we have both the block-level netlist and the corresponding subcircuits, AMSgen automatically references the matching circuits and joins the netlists to form the complete circuit. This process builds a fully functional netlist for simulation and prepares us for the upcoming sizing step. Currently, AMSnet contains only two block-level netlists, one for differential-input single-output OPAMPs and another for fully differential OPAMPs. As we expand into larger circuit types, there will be more block-level circuits to choose from. We will then resort to knowledge graph retrieval, similar to how we retrieve the subcircuits, to select the appropriate block-level netlists.

### 3.3.2 Constraint-Augmented Sizing

Now that the circuit and test benches are fixed, the next step is to optimize component parameters to achieve maximum performance with the given topology. Sizing is formulated as a black-box optimization problem and is resolved through Bayesian optimization (BO) with local annotations retrieved from AMSnet.

**Problem Formulation:** We present the standard constrained optimization problem in Equation 3.1:

$$\begin{aligned} &\text{maximize} \quad FoM(x) \\ &\text{s.t.} \quad g_j(x) \leq 0, \quad \forall j \in \{1, \ldots, p\} \end{aligned} \tag{3.1}$$

where the figure of merits (FoM) is the objective function, and $g_j(x)$ is the $j$-th performance constraint. Following [43, 49], we define FoM as the weighted square sum of the normalized performance value, as shown in Equation 3.2:

$$FoM = \sum_{i=0}^{N} w_i \times \frac{\min(f_i(x), f_i^{\text{bound}}) - f_i^{\min}}{f_i^{\max} - f_i^{\min}} \tag{3.2}$$

where $f_i(x)$ is $i$-th simulated performance, $f_i^{\text{bound}}$ is the performance limit above which we do not get additional merit, $f_i^{\min}$ and $f_i^{\max}$ are normalization factors obtained through an initial set of random samples, and $w_i$ is the term weight to control the importance of each performance metric.

Bayesian optimization (BO) is a strategy for global optimization, particularly suitable for optimizing black-box functions. It is usually employed when the function evaluation is expensive or the search space is too large to exhaust. BO treats the black-box function to be optimized as a stochastic process, typically modeled using a Gaussian Process (GP) as a surrogate model. The GP provides a flexible way to describe the distribution of the black-box function and updates predictions based on existing data points.

By sampling initial data points, the GP model is trained to fit the black-box function. An acquisition function is selected to balance exploration and exploitation, determining the next sampling point. In the input space, the point that maximizes the acquisition function is found. The selected new point is then evaluated through simulation, and its result is added to the existing dataset, updating the GP model. By balancing exploration and exploitation, BO can avoid getting trapped in local optima and find the global optimum.

**Gaussian Process:** A GP is a stochastic process defined over an input space, where any finite subset of random variables follows a multivariate Gaussian distribution. Given a training dataset $\{(x_i, y_i)\}_{i=1}^n$, where $x_i$ are the inputs and $y_i$ are the target values. A GP is completely defined by its mean function $\mu(x)$ and covariance function $k(x, x')$. The mean function represents the expected value of the function at a given input, and the covariance function (also called the kernel function) represents the correlation or similarity between any two points. Generally, the mean function of a GP is assumed to be zero:

$$m(x) = \mathbb{E}[f(x)] = 0 \tag{3.3}$$

The kernel function $k(x, x')$ is defined as:

$$k(x, x') = \mathbb{E}[(f(x) - m(x))(f(x') - m(x'))] \tag{3.4}$$

For a new input point $x_*$, the predictive distribution is also a Gaussian distribution. The mean and variance of the prediction can be computed as follows:

$$\mu_* = k(x_*, X)^T K^{-1} y \tag{3.5}$$

$$\sigma_*^2 = k(x_*, x_*) - k(x_*, X)^T K^{-1} k(x_*, X) \tag{3.6}$$

where $k(x_*, X)$ is the covariance vector between the new input point and all training data points, and $y$ is the vector of target values for the training data.

**Acquisition Function:** The acquisition function is a key component in BO, used to select the next evaluation point. It balances exploration and exploitation, making trade-offs between **exploring** unknown regions to gain knowledge and **exploiting** the best-known regions to find optima.

**Expected Improvement (EI)** : EI measures the expected amount of improvement over the current best point. The formula is:

$$\alpha_{EI}(x) = \mathbb{E}[\max(0, f(x) - f(x^+))] \tag{3.7}$$

where $f(x^+)$ is the current best observed value.

By modeling our custom FoM as a black-box function of the design parameters, we are able to use a GP to fit the simulation and use BO to optimize our parameters without explicit knowledge.

**Sizing Constraints:** Exhaustively searching parameters for all transistors is inefficient; therefore, expert insights stored in local annotations serve as *constraints* to reduce the parameter search space. For instance, with the annotation that a set of transistor parameters should maintain equality or a certain ratio, they share the same parameter with some constant multiplier to match the ratio. The algorithm flow for constraint-augmented sizing is as follows:

### 3.3.3 Topology Regeneration

In the case where an initial topology design does not satisfy the desired metrics after a set amount of sizing effort, it is considered a topology issue, and the design flow proceeds to a topology regeneration step. The topology regeneration process is very similar to the initial topology design process. The only difference is that, instead of using a different circuit application in the few-shot examples, here the actual performance specifications obtained from the previous designs are used to give the LLM additional quantitative knowledge.

With the previous design strategy proposed by the LLM and the optimal performance during simulation, the design flow essentially reverses the cause-effect and pretends that it initially *wanted* to achieve the performance metrics, and then *correctly* obtained the previous design. For example, suppose that an OPAMP with a DM gain of 60 dB is initially required, the LLM initially proposed a design strategy to retrieve circuit components and assemble circuit $A$. Then the process outlined in section 3.3.2 is applied, and finally ended with a fully sized design that only achieves a DM gain of 40 dB. AMSgen would now inject a fewshot example where it *desires a design that achieves a DM gain of 40 dB, and then obtained*

**inputs:** initial data size $N_{init}$, number of iterations $N_{iter}$, sizing constraint $C$;

$x_{init} \leftarrow \text{sample}(N_{init}, C)$;

$y_{init} \leftarrow$ performance output from simulation with $x_{init}$;

fit $\mathcal{GP}(x_{init}, y_{init})$;

**for** $t \leftarrow 0$ **to** $N_{iter}$ **do**

// select next points for simulation via acquisition function $\alpha(x)$ and sizing constraint

$x_{next} \leftarrow \arg\max_x \alpha(x)$;

$y_{next} \leftarrow$ performance output from simulation with $x_{next}$;

**if** $y_{next}$ *does not meet hard constraints* **then**

fit $\mathcal{GP}(x_{next}$, -INF);

**else**

fit $\mathcal{GP}(x_{next}, y_{next})$;

**end**

**end**

**return** optimal sizing $x^*$;

**Algorithm 2:** Constraint-Augmented Sizing for Analog Circuit Design

*circuit A*; and then it prompts the LLM for a new design that achieves 60 dB instead. The LLM would then understand to propose a topology with a higher expected gain.

This approach is especially effective since initially the state-of-the-art LLM does not possess sufficient quantitative knowledge. It is unable to accurately infer the performance of a specific circuit topology on a specific technology node, and therefore generally begins with a balanced design that has no particular weakness. However, it does possess qualitative and comparative knowledge between different topologies, as they generally hold across all technology nodes. In this example where we require a higher gain, it is often able to correctly suggest a new topology that specializes in higher gain. With the new topology, the netlist retrieval, assembly, and sizing process are replayed for the same set of performance specifications. This topology regeneration process is repeated until either the intended performance goals are achieved, or all possible topologies are exhausted. In each case, AMSgen either finalizes the design or labels the input as impossible to achieve.

## 3.4 Experiments

We employ YOLO-V8 for object detection during the construction of AMSnet, use Neo4j for knowledge graph implementation, and utilize GPT-4 for various inquiries throughout dataset construction and circuit design. All code is implemented in Python 3.9 and executed on a CentOS 7.9.2009 system with an Intel Xeon Gold 5320 CPU featuring 96 cores. Our experiments utilize SMIC 28nm technology and are performed using Cadence's Spectre simulator. Device sizing constraints, set by the PDK, limit the transistor length, width, and number of fingers to [30nm, 1$\mu$m], [100nm, 3$\mu$m], and [1, 100], respectively. During dataset construction, we manually label 30% of our collected data for semi-supervised learning, which requires approximately 60 human-hours. Subsequently, providing local and global annotations for AMSnet consumes about 120 human-hours. We implement Bayesian optimization (BO) using the Optuna [50] library and limit our total simulation count to 2000. By

distributing the workload across 20 concurrent threads, we significantly enhance processing efficiency.

### 3.4.1 OPAMP Design

**Topology Generation:** Initially, an OPAMP design is requested to achieve the following performance goals.

$$\text{Gain} > 80\,\text{dB}, \quad \text{CMRR} > 80\,\text{dB}, \quad \text{PSRR} > 80\,\text{dB},$$
$$\text{GBW} > 10\,\text{MHz}, \quad \text{PM} > 60^\circ, \quad C_L = 100\,\text{pF}$$

A comparator design flow serves as a few-shot example for generating design strategies. Fig. 5.1 and Fig. 5.2 in Appendix section 5 present the full conversation. In this instance, the LLM suggests a design strategy for a two-stage OPAMP, where the first stage is a differential amplifier with a current mirror load. The second stage is a common-source amplifier, and Miller compensation is added between the outputs of the first and second stages.

This design strategy is then converted into a set of relation query pairs in the form of ¡relation, value¿. Fig. 5.3 and Fig. 5.4 in Appendix section 5 present the full conversation. These pairs are used to query AMSnet and retrieve circuits. For example, the pair ¡input, differential input pair¿ and ¡load, PMOS current mirror¿ retrieve a single-stage 5-transistor OPAMP with a relevance score of 2, as both queries match. The circuit with the highest relevance score is selected for assembly. Similarly, a common-source amplifier is retrieved for the second stage, along with the bias circuitry featuring a current source and an NMOS transistor, and the sequentially connected resistor-capacitor pair for Miller compensation. Utilizing the predefined block-level circuit for 2-stage OPAMPs, the output from the first stage is connected to the input of the second stage, the bias voltages of both stages to the bias circuitry, and Miller compensation between the outputs of the first and second stages. The fully assembled 2-stage OPAMP is depicted in Fig. 3.5.

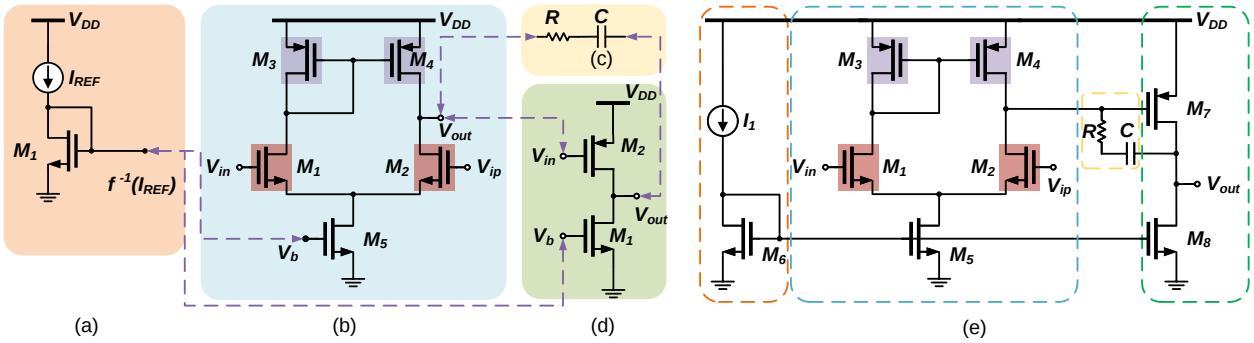



Figure 3.5: Visualization of the retrieved circuit components followed by the assembled result. The purple arrows illustrate circuit-component-level connectivity. (a) Bias circuitry, (b) First stage, (c) Miller compensation, (d) second stage, (e) Assembled 2-stage OPAMP, symmetric transistors identified from local annotations are highlighted in the same color.

**Sizing:** The specified performance metrics form the Figure of Merit (FoM) in Equation 3.8, where $f_{gain}(x)$, $f_{cmrr}(x)$, $f_{psrr}(x)$, $f_{gbw}(x)$, and $f_{pm}(x)$ represent the simulated gain, Common Mode Rejection Ratio (CMRR), Power Supply Rejection Ratio (PSRR), gain-bandwidth product, and phase margin, respectively. Each $f^{min}$ and $f^{max}$ denote the corresponding minimum and maximum metric values for normalization, observed within an initial set of 100 simulations. Notably, since the distribution of GBW values is exponential, we first take the decimal logarithm of each value before using it in the FoM to prevent it from dominating other values. Regarding the phase margin, rather than seeking a high value, we aim for the value to be as close to 60 degrees as possible, introducing its deviation from 60 degrees as a penalty.

$$
\begin{aligned}
FoM = & \frac{\min(f_{gain}(x), 80) - f_{gain}^{\min}}{f_{gain}^{\max} - f_{gain}^{\min}} + \frac{\min(f_{cmrr}(x), 80) - f_{cmrr}^{\min}}{f_{cmrr}^{\max} - f_{cmrr}^{\min}} + \frac{\min(f_{psrr}(x), 80) - f_{psrr}^{\min}}{f_{psrr}^{\max} - f_{psrr}^{\min}} + \\
& \frac{\min(\lg(f_{gbw}(x)), 7) - \lg(f_{gbw}^{\min})}{\lg(f_{gbw}^{\max}) - \lg(f_{gbw}^{\min})} - \frac{|f_{pm}(x) - 60|}{f_{pm}^{\max} - f_{pm}^{\min}}
\end{aligned}
\tag{3.8}
$$

As depicted in Fig. 3.5(e), our design incorporates 5 NMOS transistors and 3 PMOS

transistors, each characterized by two parameters: gate length ($L$) and gate width ($W$). Additionally, a current source, a capacitor, and a resistor are each defined by a single parameter, totaling 19 free parameters.

Insights from experienced engineers can significantly narrow the search space. The local annotations stored in AMSnet are utilized for symmetry checks. As illustrated in Fig. 3.5, transistors highlighted in the same color share identical $L$ and $W$ values. This approach reduces the effective number of free parameters from 19 to 15, making it more manageable.

Fig. 3.6 illustrates the optimization trajectory for the initial design, comparing *constraint-augmented sizing* with *vanilla sizing*. It is evident that a smaller number of free parameters leads to earlier convergence. The final designs with the highest Figure of Merit (FoM) are presented in Table 3.1 rows (a) and (b) for the two attempts. Notably, attempt (b) significantly outperforms attempt (a), demonstrating that the retrieved local annotations not only improve final performance but also reduce convergence time.

Unfortunately, in this initial topology, the optimal sizing achieved through Bayesian optimization fails to meet our specification requirements. The gain value reaches only 66.21 dB, substantially below the required 80 dB. Additionally, the Common Mode Rejection Ratio (CMRR) and Power Supply Rejection Ratio (PSRR) also fall short of expectations. While it is possible that our sizing step did not achieve the global optimal set of parameters for this topology, a more effective approach might be to regenerate the circuit topology.

Table 3.2: OPAMP Initial Design presents the computational resources consumed, quantified in LLM prompt and output tokens for conversations, as well as CPU time for sizing. CPU time consumed during knowledge graph query, circuit assembly, and various data processing steps is negligible.

**Topology Regeneration:** The regeneration process for the LLM to propose another design strategy mirrors the initial strategy proposal closely. However, rather than relying solely on performance specification requirements and design structures from other applications like

| Topology & Sizing Constraints | FoM ↑ | DM gain (dB)↑ | GBW (Hz)↑ | CMRR (dB)↑ | PSRR (dB)↑ | Phase margin (°) | Power (W)↓ |
|---|---|---|---|---|---|---|---|
| Specification | - | 80.00 | $1.00 \times 10^7$ | 80.00 | 80.00 | 60.00 | - |
| (a) 2-stage W/o local annotations | 2.73 | 60.06 | $3.91 \times 10^7$ | 17.15 | 34.47 | 73.15 | $1.703 \times 10^{-3}$ |
| (b) 2-stage W/ local annotations | 3.40 | 66.21 | $\mathbf{3.19 \times 10^8}$ | 54.20 | 69.82 | 63.63 | $2.811 \times 10^{-3}$ |
| (c) 2-stage high-gain W/o local annotations | 2.55 | 45.95 | $5.05 \times 10^7$ | 20.62 | 59.89 | 63.24 | $\mathbf{9.218 \times 10^{-4}}$ |
| (d) 2-stage high-gain W/ local annotations | **3.60** | **80.85** | $1.43 \times 10^7$ | **99.04** | **91.76** | **60.48** | $1.456 \times 10^{-3}$ |

Table 3.1: Designs optimized by Bayesian optimization

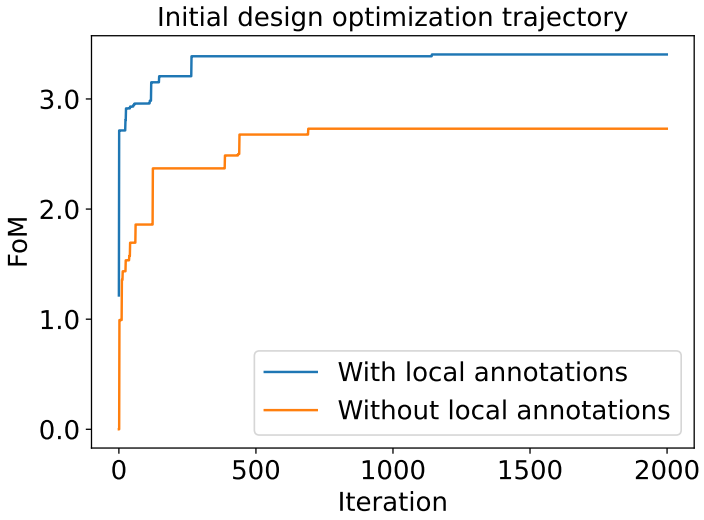



Figure 3.6: Bayesian optimization trajectory

comparators in few-shot examples, the design flow now incorporates the previous, unsuccessful design as a reference. This adaptation allows for a numerical comparison between the two circuits concerning specific specifications, thereby leveraging the LLM's extensive knowledge of comparative performance between circuits. As depicted in Fig. 5.5 and Fig. 5.6 in the Appendix section 5, the AMSgen flow inversely manipulates the cause-effect order by simulating an initial desire for a design to achieve a DM gain of 66.21 dB, gain-bandwidth product of 319MHz, etc. The analysis and design strategy are maintained as they were for the previous circuit topology. Subsequently, by posing the same inquiry, the LLM naturally obtains a quantitative comparison between the performance specification in the few-shot example and the actual prompt. This approach effectively addresses the previous issue where the LLM lacked specific knowledge about the technology node.

| Experiment | LLM cost (input tokens, output tokens) | | | Sizing cost (CPU seconds) |
|---|---|---|---|---|
| | Design Strategy | Relation Pair | Tie Breaker | |
| OPAMP Initial Design | 516 — 407 | 626 — 101 | N/A | 678.5 |
| OPAMP Revised Design | 590 — 476 | 693 — 353 | N/A | 739.7 |
| Comparator Design | 665 — 423 | 411 — 151 | 37 — 253 | 640.0 |

Table 3.2: Experiment cost for case studies

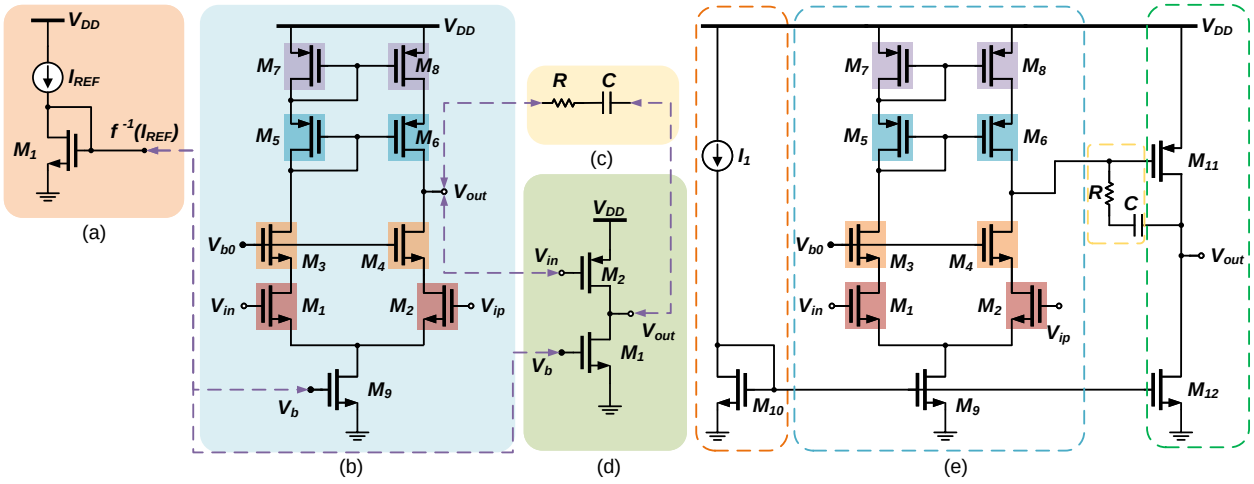



Figure 3.7: Visualization of the retrieved circuit components followed by the assembled result. The purple arrows illustrate circuit component -level connectivity. (a) Bias circuitry, (b) First stage, (c) Miller compensation, (d) second stage, (e) Regenerated 2-stage OPAMP. Symmetric transistors are highlighted in the same color.

In our scenario, since the differential mode (DM) gain was insufficiently low, a natural response from an engineer would be to opt for a different topology that specializes in achieving higher DM gain. Mirroring this reasoning, the LLM decides to update the first stage from the existing five-transistor configuration to a telescopic cascode topology while maintaining the rest of the design unchanged. This updated information is then employed in the same AMSnet retrieval process as before, followed by an assembly process informed by local annotations. The resulting circuit is depicted in Fig. 3.7.

The sizing process remains largely unchanged from the previous iteration, with the pri-

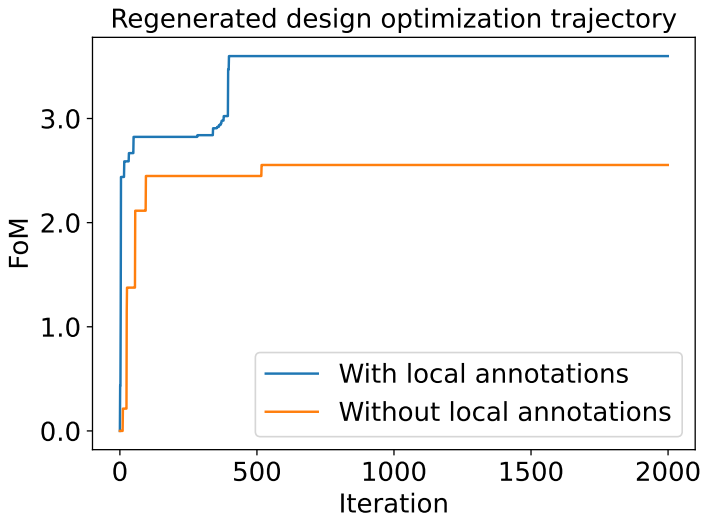



Figure 3.8: Bayesian optimization trajectory for the regenerated design of higher DM gain

mary difference being the addition of several new transistors and a bias voltage. Utilizing symmetry constraints from local annotations, the number of free parameters is reduced from 27 to 19. As depicted in Fig. 3.8, the optimization trajectory mirrors the shape of the previous topology's trajectory. However, quantitatively, the Figure of Merit (FoM) score achieved is significantly improved, successfully meeting all design goals. Nonetheless, convergence is achieved later due to the increased number of parameters and the more stringent specifications. Additionally, the sizing attempt without parameter constraints takes considerably longer to improve the FoM and fails to produce an acceptable design within the 2000 iteration limit.

The final performance, detailed in Table 3.1 row (d), indicates that every design goal initially set was achieved, confirming the success of the design flow.

### 3.4.2 Comparator Design

To evaluate the AMSgen workflow across different applications, we introduce a case study focusing on comparators. Like the process for generating OPAMPs, AMSgen starts by

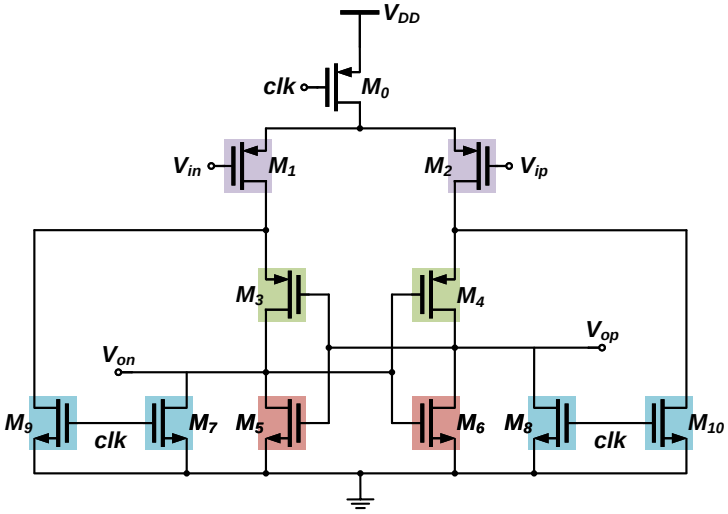



Figure 3.9: Visualization of the retrieved strong-arm latch comparator. Symmetric transistors are highlighted in the same color.

requesting a comparator design from LLMs, using the same PDK and constraints. The comparator-specific set of performance specifications is defined as follows:

$$\text{Sampling frequency} = 1\text{GHz}, \quad \text{Offset voltage} < 100\,\text{uV}$$

$$\text{Propagation delay} \leq 1\,\text{ns}, \quad \text{Power} \leq 100\,\text{uW}$$

Due to the fewer comparator topologies and the recyclability of subcircuits, AMSnet stores full comparator topologies rather than subcircuits and block-level circuits, as is the case with OPAMPs. Initially, the LLM recommends a latch comparator. This response is then transformed into relation query pairs, similar to the OPAMP case study, and a search is conducted within AMSnet for matching topologies.

This time, two different types of comparator topologies are retrieved: the strong-arm latch comparator and the double-tail latch comparator. To break the tie, the names of these two topologies are fed back into the LLM to make a final selection, which results in the choice of the strong-arm latch comparator. The detailed conversations regarding this selection process are presented in the Appendix section 5 from Fig. 5.9 through Fig. 5.13.

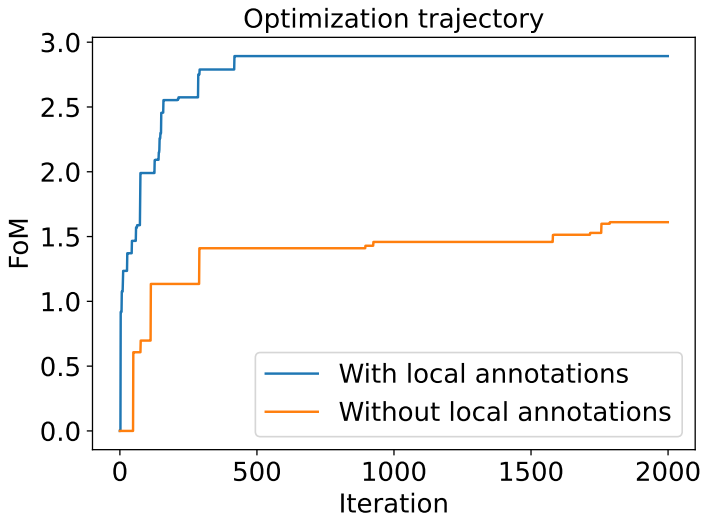



Figure 3.10: Bayesian optimization trajectory for strong-arm latch comparator

$$FoM = -\frac{\max(\lg(f_{ov}(x)), -4) - \lg(f_{ov}^{\max})}{\lg(f_{ov}^{\max}) - \lg(f_{ov}^{\min})} - \frac{\max(\lg(f_{pd}(x)), -9) - \lg(f_{pd}^{\max})}{\lg(f_{pd}^{\max}) - \lg(f_{pd}^{\min})} - \frac{\max(\lg(f_{p}(x)), -4) - \lg(f_{p}^{\max})}{\lg(f_{p}^{\max}) - \lg(f_{p}^{\min})} \tag{3.9}$$

Similar to the OPAMP case study, Bayesian optimization is employed to obtain the optimal sizing using the FoM in Equation 3.9, where $f_{ov}(x)$, $f_{pd}(x)$, and $f_p(x)$ represent the simulated offset voltage, propagation delay, and power for the given sizing $x$. Each metric is viewed on a logarithmic scale, akin to GBW in the OPAMP study. A similar set of local annotations is used to guide sizing constraints. As shown in Fig. 3.9, there are a total of 5 PMOS and 6 NMOS transistors, each parameterized by length and width, totaling 22 parameters. After considering symmetry, the number of free parameters drops to 10.

The optimization trajectory is displayed in Fig. 3.10. As in the OPAMP case study, the presence of local annotations significantly enhances both convergence time and final performance. As indicated in Table 3.3(b), all desired performance specifications are met. This case study serves as another data point to demonstrate the generality of AMSgen.

| **Topology & Sizing constraint** | **FoM ↑** | **Offset voltage (V)↓** | **Propagation delay** (s)↓ | **Power (W)↓** |
|---|---|---|---|---|
| Specification | - | $1.00 \times 10^{-4}$ | $1.00 \times 10^{-9}$ | $1.00 \times 10^{-4}$ |
| (a) Strong-arm W/o local annotations | 1.61 | $7.98 \times 10^{-4}$ | $5.81 \times 10^{-10}$ | $3.42 \times 10^{-4}$ |
| (b) Strong-arm W/ local annotations | **2.89** | $\mathbf{3.50 \times 10^{-5}}$ | $\mathbf{1.19 \times 10^{-11}}$ | $\mathbf{8.84 \times 10^{-5}}$ |

Table 3.3: Designs optimized by Bayesian optimization

### 3.4.3 Robustness Studies

To verify the robustness of AMSgen, we conduct repetition tests on our OPAMP topology generation and Bayesian optimization flow. Given identical performance specifications, the LLM-generated design strategies vary with each attempt, resulting in different topologies, as depicted in Fig. 3.11. Despite the initial variations, the topologies using a current mirror load and a resistive load in the first stage fail to meet the specification requirements and are consequently regenerated to the telescopic design. This underlines the robustness of AMSgen, as the correctness of our topology is confirmed through simulation, counteracting the randomness of the LLM.

Fig. 3.12 illustrates the Bayesian optimization trajectory for 10 repeated sizing attempts on the 2-stage telescopic cascode topology. The FoM trajectory shows a gradual convergence, with the $\pm 1$ std. dev. range (depicted in light blue) narrowing over the iterations. This indicates that while different BO attempts may vary in the number of iterations needed for convergence, the results consistently align. Additionally, it is evident that attempts without transistor sizing constraints, as identified in AMSnet through local annotations, display a noticeably lower FoM trajectory, underscoring the effectiveness and efficiency of our method.

Furthermore, to ensure that AMSgen functions with any reasonable set of specifications, we proceed with a different set of specifications focusing on low power as follows.

$$\text{Gain} > 50\,\text{dB}, \quad \text{CMRR} > 50\,\text{dB}, \quad \text{PSRR} > 50\,\text{dB},$$
$$\text{GBW} > 40\,\text{MHz}, \quad \text{PM} > 60^\circ, \quad C_L = 100\,\text{pF}, \quad \text{power} < 0.2mW$$

Our topology generation step resulted in the 2-stage OPAMP depicted in Fig. 3.5. For our

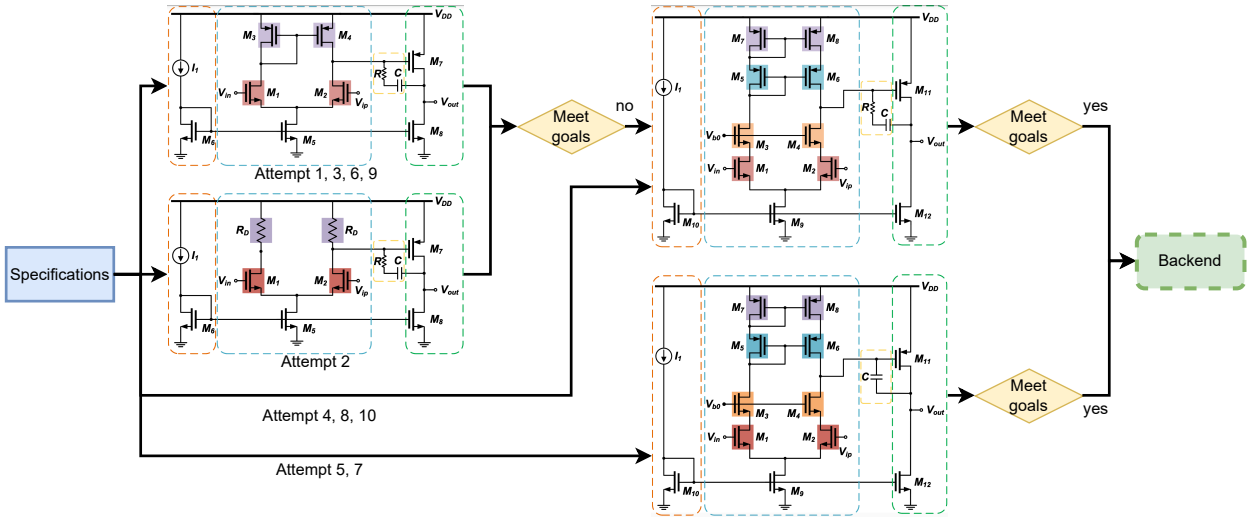



Figure 3.11: Topology generation replayed 10 times

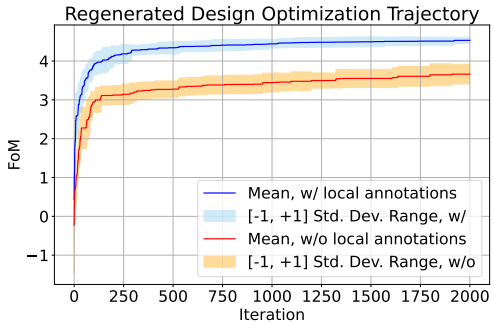



Figure 3.12: Bayesian optimization replayed 10 times

previous set of specifications requiring higher gain, this topology did not meet the requirements after the sizing step. However, it successfully satisfies the current set of specifications after 262 Bayesian optimization (BO) iterations. The resulting performance is as follows.

$$\text{Gain} = 52.86\,\text{dB}, \text{CMRR} = 71.60\,\text{dB}, \text{PSRR} = 55.29\,\text{dB},$$

$$\text{GBW} = 47.70\,\text{MHz}, \text{PM} = 60.12^\circ, C_L = 100\,\text{pF}, \text{power} = 0.11mW$$

This satisfies all performance requirements, proving that our method works on other reasonable specifications as well.

## 3.5 Discussions

In this chapter, we introduced AMSgen, a fully automated, data-driven framework for analog and mixed-signal (AMS) circuit generation, built upon the AMSnet dataset. AMSgen leverages large language models (LLMs) to emulate the decision-making process of experienced engineers, utilizing our knowledge graph for topological retrieval and assembly. It further employs Bayesian optimization to derive optimal device sizing, completing the design flow from high-level performance specifications to fully sized netlists without any human intervention beyond the initial input.

Experimental results demonstrate AMSgen's ability to generate complex circuits such as operational amplifiers and comparators that meet all specified performance targets. These results validate the feasibility and effectiveness of our modular approach that separates topology construction from sizing and simulation, while incorporating expert knowledge through structured annotations.

Despite these promising outcomes, several challenges remain. First, the Bayesian optimization process becomes computationally intensive as circuit complexity increases. For larger systems, thousands of simulations may be required, leading to prohibitive runtime if performed naively. The next chapter addresses this by introducing a divide-and-conquer approach, using a SAR ADC design as a case study to scale AMSgen to more complex circuits.

Second, current experiments rely exclusively on pre-layout simulations, which omit parasitic effects. This limits the accuracy of the sizing results in post-fabrication conditions. However, recent advances in automated layout generation, such as abuttable analog cells (Acells) [51], offer a path toward integrating post-layout simulations into the optimization loop. Future work will incorporate automated layout generation to ensure that parasitics are accounted for during sizing.

Finally, AMSgen currently depends on preset testbench topologies for simulation, which

constrains the diversity of circuits and metrics that can be evaluated. To overcome this limitation, future work will explore LLM-driven generation of testbenches, including both topology and metric extraction code, thereby expanding the range of supported circuit types and performance evaluations.

Overall, AMSgen marks a significant step toward fully autonomous AMS circuit design, demonstrating that LLMs, when grounded in domain-specific data and augmented with structured engineering knowledge, can drive meaningful progress in analog EDA automation.

# CHAPTER 4

# Automated SAR ADC Design Using Analytic Equations

## 4.1 Introduction

Analog-to-digital converters (ADCs), connecting analog sensors to digital processing, have become an essential IP in most System on Chips (SoCs). Successive Approximation Register (SAR) ADCs, with applications ranging from medium-low precision to ultra-high-speed designs, have attracted significant attention [52, 53, 54]. However, conventional design methods primarily rely on manual adjustments. As technology nodes shrink and design complexity grows exponentially, manually designing SAR ADCs has become increasingly time-consuming. This inefficient design process struggles to meet the demands of a fast-evolving market, necessitating an automatic design methodology.

Device sizing is a crucial step in the design flow of analog and mixed-signal (AMS) integrated circuits, such as SAR ADC. Previous work on automatic sizing can be broadly classified into knowledge-based and optimization-based approaches. Knowledge-based approaches rely on analytic equations to derive circuit performance and device sizing, usually involving assumptions and approximations with various empirical values. In contrast, optimization-based approaches depend on trials and errors in the simulation environments and are very time-consuming. Significant research effort [52, 43, 55, 56] has been dedicated to automating optimization-based methods using machine learning (ML) algorithms such as Bayesian optimization (BO) and reinforcement learning (RL). However, these methods are

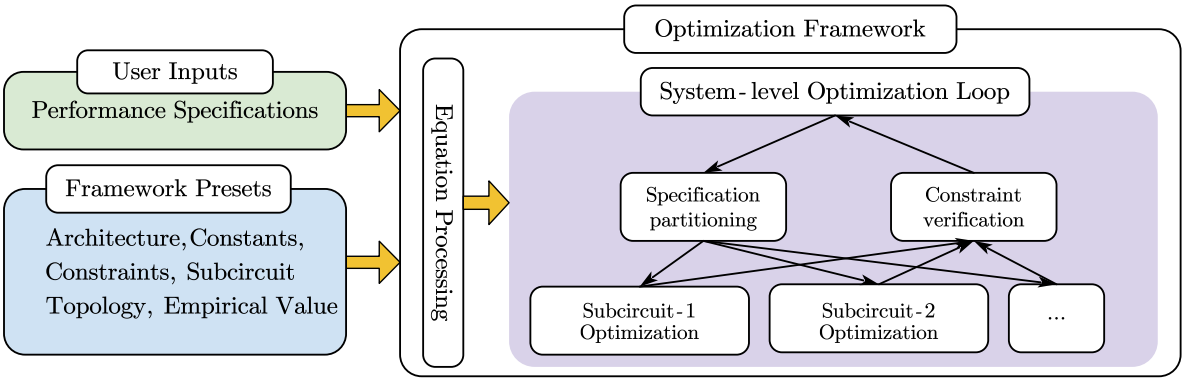



Figure 4.1: Automated Design Framework Overview

often limited to small-scale circuits, such as OPAMP or comparator designs within twenty transistors. Due to the extensive parameter search spaces and high simulation costs, only a few studies [57, 58, 59] have been extended to larger ones, such as ADCs. On the other hand, there have been some pioneering knowledge-based automatic sizing [20], but again the circuit scale is limited.

There have been research achievements on automatic sizing of SAR ADCs. However, the proposed methods typically require human interventions within the optimization loop. This demands professional SAR-ADC-specific knowledge, and could potentially introduce human errors. [57, 58] both achieve some level of automation in SAR ADC design. However, [57] still requires a human expert to determine the priority of device sizing. [58] needs a pre-characterized comparator library of different device parameters, which is time-consuming to obtain.

To address the above challenges, this chapter proposes an automated system-level design methodology for SAR ADCs, as illustrated in Fig. 4.1. The overall motivation is to make use of numerous analytical equations that maps performance from complex circuits to its simpler subcircuits, where latter are usually within reach of popular automated sizing frameworks. The tool requires only high-level performance specifications as input, and the built-in library will supply the circuit topology, testbenches, and analytical equations, facil-

itating automatic generation of complete SAR ADC netlists without human intervention. It begins with an equation processing phase, during which a dependency graph is constructed from the analytical equations to establish a topological order for simulations for verification, knowledge-based calculations and optimization-based transistor sizing. Based on the dependency graph, we integrate a dual optimization scheme to address the high-dimensional transistor sizing optimization problem in SAR ADCs. The system-level optimization iteratively decomposes the overall performance specifications into individual subcircuits, while several local optimization loops employ BO to optimize the transistor parameters of the subcircuits. The locally optimized results of the subcircuits are then fed back to the system level for overall performance simulation or calculation. Finally, the loop concludes once all constraints are satisfied, yielding the complete device sizing for the SAR ADC.

Our contributions are as follows.

- We propose a fully-automated SAR ADC design methodology that generates optimized device sizing directly from performance specifications. The dependency graph-based framework is proposed to serialize the simulations for verification, knowledge-based calculations, and transistor sizing optimization in topological order.
- We utilize a dual optimization scheme to tackle the high-dimensional sizing problem. The system-level optimization iteratively decomposes the overall performance specifications into individual subcircuits, while local optimization loops employ BO to optimize the transistor parameters of the subcircuits. This ensures that both overall performance goals and individual subcircuit constraints are met efficiently.
- We demonstrate the effectiveness of our framework through two case studies with varying performance specifications.

## 4.2 Preliminaries

This section introduces the background knowledge of SAR ADCs including the key design considerations. We also provide a brief overview of Bayesian optimization (BO), the machine learning algorithm employed to optimize device parameters and empirical values.

### 4.2.1 SAR ADC Architecture

Fig. 4.2 shows a typical SAR ADC architecture. The functionality of each subcircuit is listed below.

- **Bootstrap Sample-and-Hold Switch ($\mathbf{SW_{S/H}}$):** Captures and stores the input voltage during the sampling phase to provide a stable input signal for conversion.
- **Capacitive Digital-to-Analog Converter (CDAC):** Converts the digital code from the SAR logic into an analog voltage for comparison.
- **Pre-amplifier:** Amplifies the input signal to a desired level, enhancing accuracy and minimizing noise before comparison.
- **Comparator:** Compares the CDAC outputs and generates a digital decision.
- **SAR Logic:** Controls the iterative conversion process, adjusting the CDAC and comparator settings to produce the final digital output.

The main performance metrics of the SAR ADC usually include resolution $N$, sampling frequency $f_s$, power consumption, and area.

### 4.2.2 Bayesian Optimization

Eq. 4.1 describes an abstract optimization problem, where we seek to maximize a certain objective function $F(\mathbf{x})$ by tuning the input variables $\mathbf{x}$, subject to a series of $p$ constraints

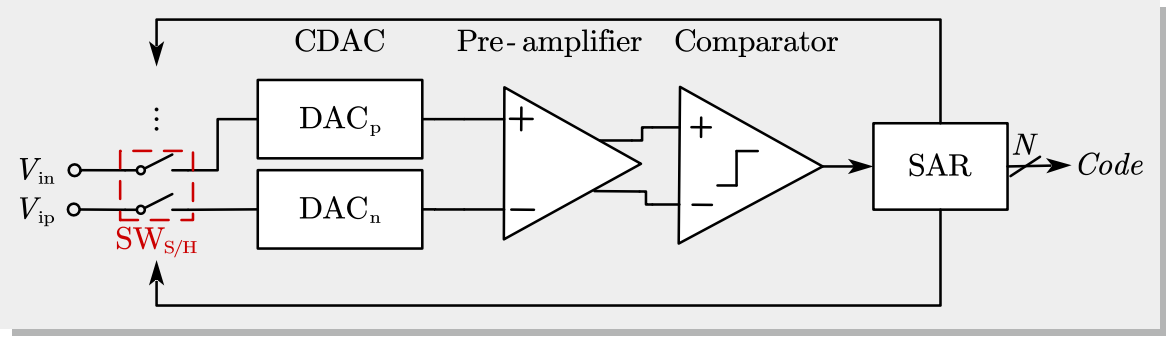



Figure 4.2: A typical SAR ADC architecture, which is assumed in this work

.

$g_j(\mathbf{x}) \leq 0$. In the context of device sizing, $F(\mathbf{x})$ is typically defined by some performance metrics, where each $g_j(\mathbf{x})$ describes the legal search space of a certain parameter, usually set by the PDK or prior knowledge.

$$\begin{aligned} &\text{maximize} \quad F(\mathbf{x}) \\ \text{s.t.} \quad &g_j(\mathbf{x}) \leq 0, \quad \forall j \in \{1, \ldots, p\} \end{aligned} \tag{4.1}$$

Bayesian optimization uses a machine learning model, typically a Gaussian process, as the surrogate model to represent the true relationship between the input parameters and the output objective function. The optimization process is iterative; in each iteration, it interacts with the unknown function $F(\mathbf{x})$ with some input $\mathbf{x}$ carefully chosen by its acquisition function, and uses the output to update the surrogate model. After sufficient iterations, it is expected to converge to a sufficiently strong objective.

## 4.3 Algorithms

In this section, we introduce the algorithms in the proposed optimization framework shown in Fig. 4.6. Due to its complexity, device sizing for SAR ADCs cannot be achieved holistically. A divide-and-conquer design approach is required to decompose the problem into

Table 4.1: Variables and Relations

| | Variables | Relations |
|---|---|---|
| **System-Level** | $\mathbf{N}$, $\mathbf{F_s}$, $\mathbf{V_{fs}}$, $\mathbf{V_{DD}}$ $\mathbf{R_{on,TG}}$ | $\mathbf{V_{DD}} = 0.9$, $\mathbf{R_{on,TG}} = 150$ |
| **Empirical Values** | $\mathbf{D}$, $\mathbf{E}$, $\mathbf{PSA}$, $\mathbf{PSTR}$ | $1 \leq \mathbf{D} \leq 3$, $1 \leq \mathbf{E} \leq 3$, $0.5 \leq \mathbf{PSA} \leq 0.9$, $0.5 \leq \mathbf{PSTR} \leq 0.9$ |
| **Bootstrap** | $\mathbf{ENOB}$, $\mathbf{C_L}$, $\mathbf{R_{on}}$, $\mathbf{R_{on,max}}$ | $\mathbf{ENOB} \geq N + D, \mathbf{C_L} \geqslant \left(6 \cdot 2^{2N-2} kT\right) / \left[V_{\text{fs}}^2 \left(10^{\frac{1}{10}} - 1\right)\right]$, $\mathbf{R_{on,max}} = max\left\{R_{on,\text{TG}}, R_{on.\text{BS}}\right\}$ |
| **CDAC** | $\mathbf{C_u}$, $\sigma_\mathbf{u}$, $\sigma_{\mathbf{u,constraint}}$, $\mathbf{C_{L,max}}$ | $\mathbf{C_u} = \frac{C_L}{2^{N/2}}$, $\sigma_\mathbf{u} \leqslant \sigma_{u,\text{constraint}}$, $\sigma_{\mathbf{u,constraint}} = \frac{C_u}{6\sqrt{2^N - 1}}$ $\mathbf{C_{L,max}} = \left(2^{N/2} - 1\right) C_u$ |
| **Preamplifier** | $\mathbf{A_V}$, $\mathbf{f_{-3dB}}$ | $\mathbf{T_{comp,constraint}} = 1.2 R_{on,\max} C_{L,\max} \ln\left(2^{N+E}\right)$, $\mathbf{A_V} \geqslant 20 \log_{10}\left(V_{os} 2^{N+1}\right)$, $\mathbf{f_{-3dB}} \geqslant \frac{\ln(1-\text{PSA})}{-2\pi \text{PSTR} \cdot T_{comp}}$ |
| **Comparator** | $\mathbf{T_{comp,constraint}}$, $\mathbf{T_{pd}}$, $\mathbf{V_{os}}$, $\mathbf{T_{comp}}$ | $\mathbf{T_{pd}} \leqslant 0.2 T_{\text{comp,constraint}}$, $\mathbf{T_{comp}} = \frac{1}{f_s} \frac{1}{(N+n)} \geqslant T_{\text{comp,constraint}}$ |

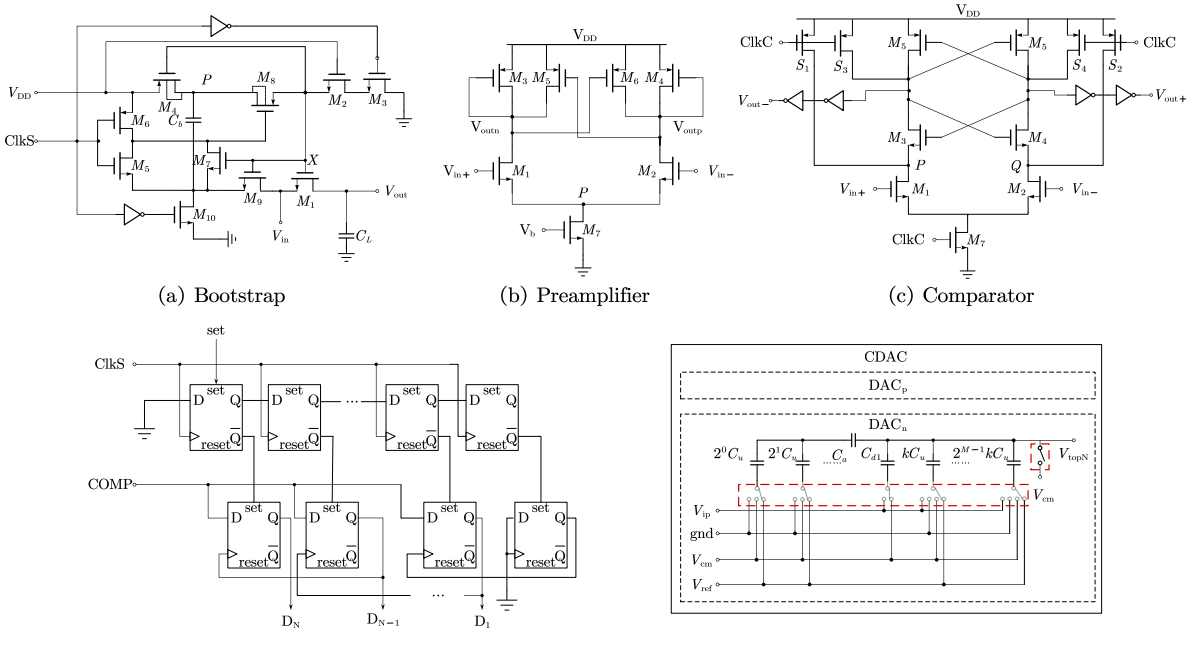



Figure 4.3: Circuit topologies of SAR ADC subcircuits

subcircuit-level challenges, making them easier to address. This is accomplished through metric mapping, where the performance specifications of the complete design are analytically translated into the specifications for each individual subcircuit. Each subcircuit is then optimized independently and, once finalized, integrated into the full SAR ADC for performance verification.

Section 4.3.1 first formulates the problem with the overall and subcircuit-level performance requirements into **variables** and **relations**. Section 4.3.2 presents the **dependency graph** used to first partition specifications to subcircuits, then serialize the analytical calculation, subcircuit optimization, and constraint verification steps. Section 4.3.3 describes how BO is applied in two distinct contexts: **subcircuit-level** BO to optimize individual subcircuits and **system-level** BO to optimize the overall design.

### 4.3.1 Subcircuit Design Specifications

In the architecture shown in Fig. 4.2, the subcircuits we assume are presented in Fig. 4.3. The optimization objectives and design constraints are summarized in Table 4.1. Subcircuit performance metrics are constrained not only by overall performance goals but also by interdependencies among them. For clarity, we define a **variable** as any constant (e.g., the Boltzmann constant $k$), empirical value (e.g., selecting an *opamp setup accuracy* between 0.5 and 0.9), overall performance goal (e.g., ADC resolution $N$), or subcircuit performance goal (e.g., preamplifier gain). Similarly, we define a **relation** as any equation or inequality constraint. With this in mind, design considerations of subcircuits are presented as follows.

**Bootstrap sampling switch:** Fig. 4.3(a) presents the bootstrap circuit. The load capacitance $C_L$ is determined by the noise constraint given by

$$C_L \geqslant \frac{6 \cdot 2^{2N-2} \cdot kT}{V_{\text{fs}}^2 \left(10^{\frac{1}{10}} - 1\right)} \tag{4.2}$$

where $k$ is the Boltzmann constant, $T$ is the absolute temperature, and $V_{\text{fs}}$ is the input amplitude.

The effective number of bits (ENOB) is an optimization objective which must satisfy the following constraint

$$\text{ENOB} \geqslant \text{N} + \text{D} \tag{4.3}$$

where D is an empirical value.

**CDAC:** Fig. 4.3(e) presents the segmented capacitive array of the CDAC. The unit capacitance and standard deviation are denoted by $C_u$ and $\sigma_u$, respectively, with the following constraint:

$$\frac{\sigma_u}{C_u} < \frac{1}{6\sqrt{2^N - 1}} \tag{4.4}$$

**Preamplifier:** The preamplifier is a three-stage cascaded structure, where a single stage is shown in Fig. 4.3(b). The required gain $A_V$ of the preamplifier is determined by the comparator offset voltage $V_{\text{os}}$ and $N$, as shown below.

$$A_V \geqslant 20 \log_{10} \left( V_{os} 2^{N+1} \right) \tag{4.5}$$

Additionally, the preamplifier bandwidth $f_{\text{-3dB}}$ is constrained by the comparator comparison time $T_{\text{comp}}$, as shown in Fig. 4.5, where the preamplifier needs to amplify the input signal within a portion of $T_{\text{comp}}$. We define this portion as the preamplifier setup time ratio PSTA, expressed as a fraction of the comparator clock cycle, with a range of [0.5, 0.9]. On the other hand, the preamplifier setup accuracy PSA refers to the accuracy of the preamplifier's output value within the setup period, with a range of [0.5, 0.9]. The constraint works out to be:

$$f_{-3\text{dB}} \geqslant \frac{\ln\left(1 - \text{PSA}\right)}{-2 \cdot \pi \cdot \text{PSTR} \cdot T_{\text{comp}}} \tag{4.6}$$

**Comparator:** Fig. 4.3(c) is a StrongARM latch comparator widely used in SAR ADCs due to its high speed, accuracy, and noise immunity. The main performance metrics for the comparator design are the comparison period $T_{\text{comp}}$, comparator propagation delay $T_{\text{pd}}$, and the offset voltage $V_{os}$. The propagation delay $T_{\text{pd}}$ is shown in Fig. 4.4, it represents the time interval from the start of the comparator's comparison to the start of the SAR logic operation. The comparison must complete within this time.

For the comparator offset $V_{os}$, to improve the accuracy of the comparator, a preamplifier is typically added, which amplifies the input signal, allowing the comparator to more accurately detect the difference between the input signal.

**SAR logic and clock control:** presented in Fig. 4.3(d). SAR logic and clock control are crucial for SAR ADC performance, impacting accuracy, speed, power consumption, and stability. SAR logic controls the successive approximation process, while clock control ensures precise timing for sampling and comparison, making optimization essential for high-efficiency, high-accuracy ADCs.

As shown in the Fig. 4.5, $T_{\text{sample}}$ represents the sampling time, $T_1$ and $T_2$ are the timing for the bottom-plate sampling, $T_{\text{h}}$ represents the hold time, and $T_{\text{comp}}$ is the comparison period of the comparator. In this paper, we set the sampling time $T_{\text{sample}}$ equal to one comparison period $T_{\text{comp}}$.

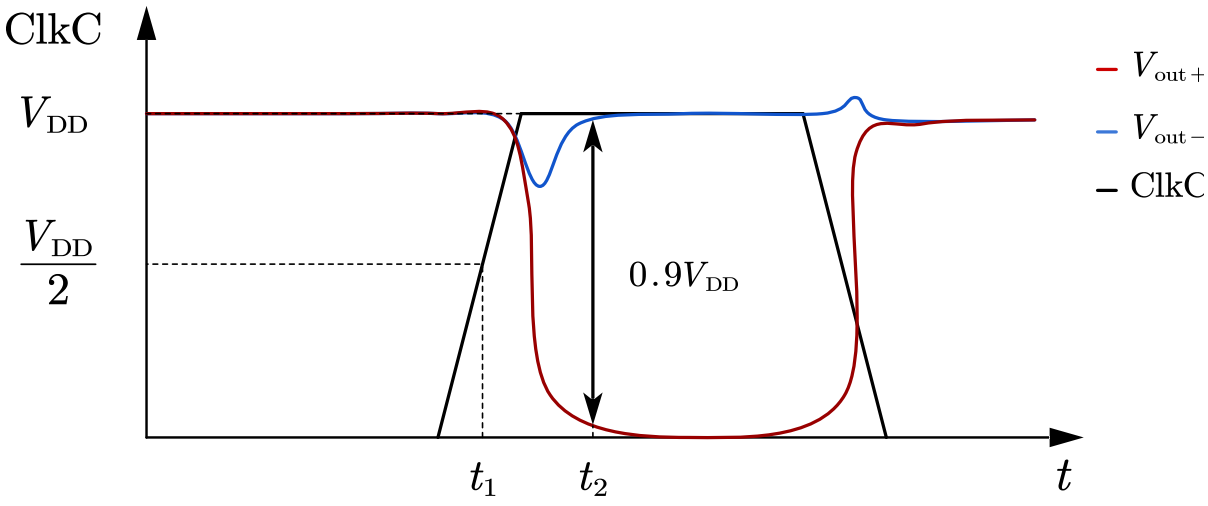



Figure 4.4: Comparator propagation delay definition: $t_2 - t_1$

For the sampling time and hold time, it is necessary to ensure that the CDAC settles with sufficient accuracy during this period. For the hold time, since bottom-plate sampling is used, the CDAC's settling time and the preamplifier's transfer time must be considered.

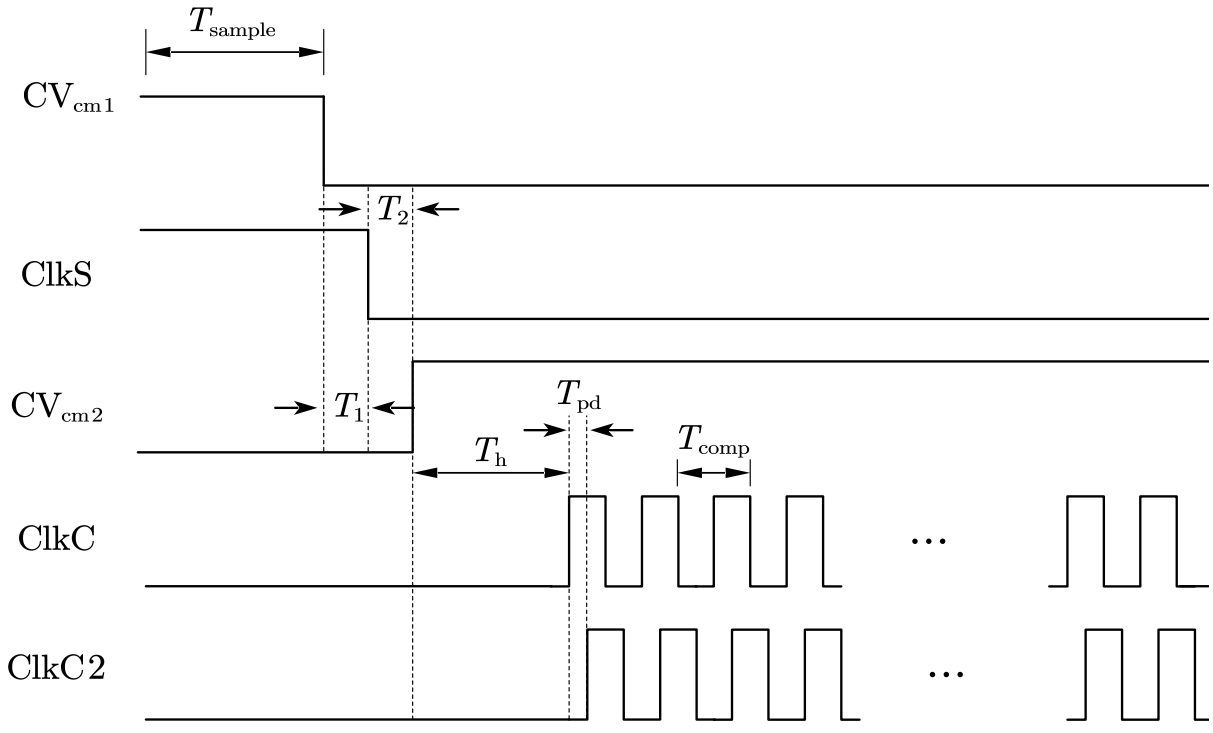



Figure 4.5: Timing diagram of the clock signals

For the hold time and the comparator's comparison period, which is an integer multiple of the entire quantization cycle, it is required to ensure that the CDAC completes the settling stage during the time interval $T_{\text{comp}} - T_{pd}$ after the comparison.

For the CDAC, other than the input signal, all other signals are controlled by transmission gate switches. When the switches are on, the switch and CDAC can be viewed as a low-pass filter, thus the constraints are:

$$T_{\text{comp}} = \frac{1}{f_s} \times \frac{1}{N+n} \geqslant 1.2 R_{\text{on,max}} C_{L,\text{max}} \ln\left(2^{N+E}\right) \tag{4.7}$$

$$T_h \geqslant 1.5 t_0 = 1.5 R_{\text{on,max}} C_{\text{tot}} \ln\left(2^{N+E}\right) \tag{4.8}$$

$$T_{\text{pd}} \leqslant \frac{T_{\text{comp}}}{5} \tag{4.9}$$

where $R_{\text{on,max}} = \max\{R_{\text{on},BS}, R_{\text{on},TG}\}$, with $R_{\text{on},BS}$ being the on-resistance of the bootstrap sampling switch, and $R_{\text{on},TG}$ being the on-resistance of the transmission gate switch.

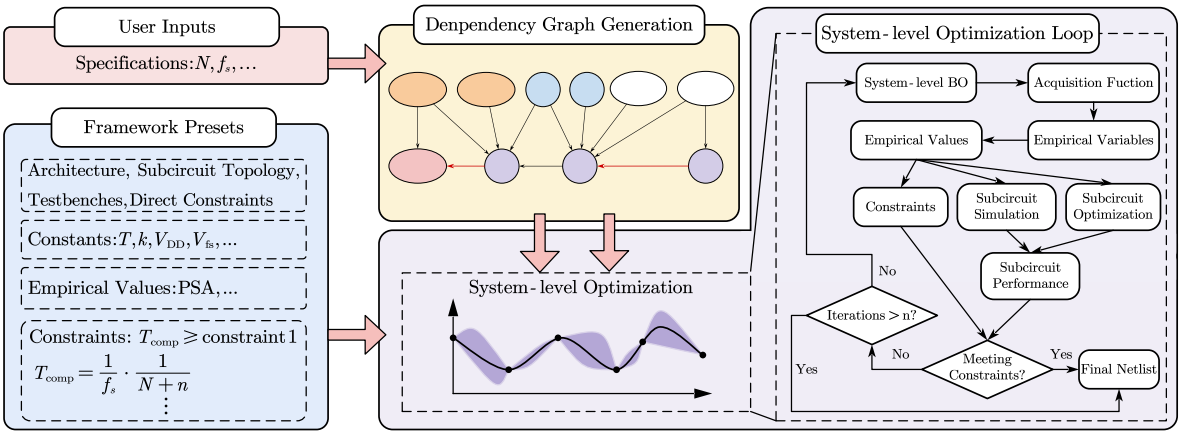



Figure 4.6: The flowchart of the proposed automated SAR ADC design methodology

$C_{L,\max}$ is the capacitance of the MSB capacitor, representing the maximum load capacitance connected to the switch during this phase. $E$ is an empirical value in the range of [1,3]. $C_{\text{tot}}$ represents the total capacitance connected to the common-mode voltage $V_{cm}$ during the hold phase $T_{\text{h}}$.

### 4.3.2 Dependency Graph Construction

Now that we have all the relevant variables and relations, we are ready to use this information for our optimization framework. As shown in Table 4.1, each variable may appear on either side of a relation; however, we ensure that only a single variable appears on the left-hand side (LHS). This establishes a **dependency** from the LHS variable to all variables on the right-hand side (RHS). By considering variables as **nodes** and dependencies as **edges**, we construct a **dependency graph** (see Fig. 4.7 for a partial example). In this work, to simplify the problem, we assume that the dependency graph is cycle-free. Consequently, the graph satisfies the properties of a *directed acyclic graph (DAG)*. This property enables topological sorting, which guarantees that all RHS variables are evaluated before their corresponding LHS expressions are computed or verified.

A subcircuit's performance specifications are derived once all related constraints are

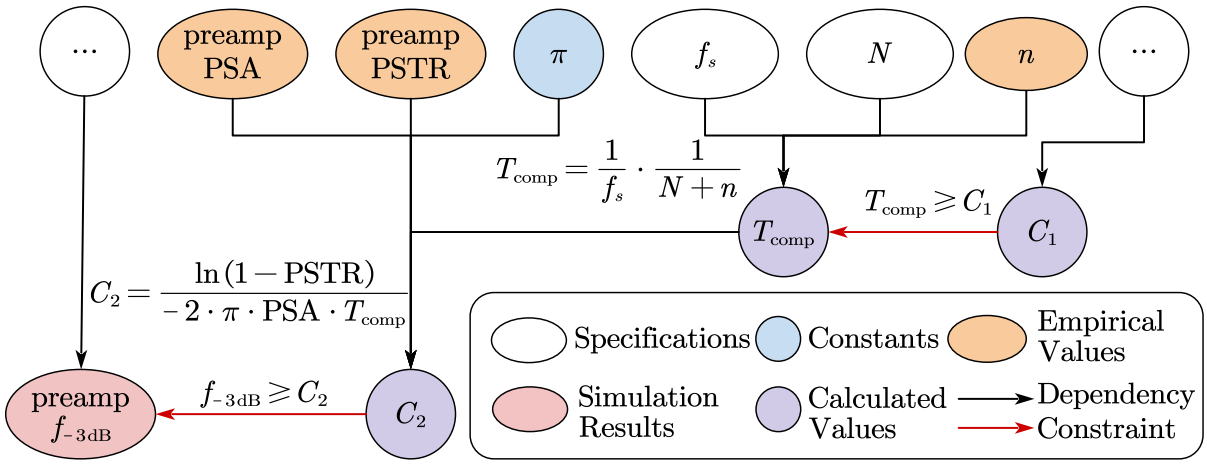



Figure 4.7: Dependency graph example, variables and equations are explained in detail in Fig. 4.6

evaluated through dependency graph propagation. This effectively partitions the system-level performance specifications to subcircuits. The flow then performs *subcircuit-level* BO on the subcircuit individually to meet the corresponding specifications. After obtaining simulation results for constraint verification, the flow propagates the performance metrics down the dependency graph in topological order to compute constraints for other subcircuits. For instance, as described in Section 4.3.1, the comparator offset $V_{os}$ is used to set a constraint on the preamplifier gain $A_V$. In this way, subcircuit-level optimization, simulation, and constraint verification are serialized.

### 4.3.3 Optimization Procedure

As shown in Fig. 4.6, BO drives the main optimization efforts in this work. After generating the dependency graph, we employ a **system-level** optimization loop to obtain a set of empirical values that balance performance and feasibility. The loop terminates when either all constraints are satisfied or the maximum number of iterations is reached. In practice, as demonstrated in Section 4.4, all constraints are met within a single iteration of system-level

Table 4.2: Comparison of different works

| | Huang† [57] | | Ding† [58] | | Hassanpourghad* [60] | | | Liu# [59] | | Our Work* | |
|---|---|---|---|---|---|---|---|---|---|---|---|
| Resolution | 10 | 12 | 8 | 12 | 6 | 8 | 10 | 10 | 12 | 12 | 14 |
| Sampling rate (MS/s) | 50 | 0.6 | 32 | 1 | 500 | 340 | 200 | 100 | 1 | 1 | 0.5 |
| Technology (nm) | 90 | 180 | 40 | | 65 | | | 40 | | 28 | |
| ENOB | 8.4 | 10.1 | 7.6 | 9.9 | 5.92 | 7.75 | 9.18 | 9.1 | 11.1 | 11.38 | 12.99 |
| SNDR (dB) | 52.1 | 62.7 | 47.4 | 61.1 | 37.3 | 48.4 | 57.8 | 56.3 | 68.8 | 70.3 | 79.99 |
| $FOM_S$ (dB) | 159.7 | 152.2 | 156.7 | 165.8 | 153.2 | 159.6 | 167 | 166.8 | 176.0 | 168.7 | 172.1 |
| $FOM_W$ (fJ/c.step) | 26.7 | 500 | 30.7 | 18.4 | 42.9 | 35.5 | 18.5 | 10.8 | 4.3 | 27.3 | 37.8 |
| TOTAL TIME | - | - | - | - | - | - | - | 1h52m42.8s* | 2h1m27.2s* | 1h2m44s | 1h8m31s |

-Unreported value. †Tapeout measurement. #Simulated results with layout R+C+CC parasitic extraction. *presimulation results.

optimization.

The **subcircuit-level** BO directly tunes device sizing (e.g. transistor length $l$, width $w$) to optimize performance. The allowed sizing range is given by the PDK. If any subcircuit performance metric fails to meet its constraint, the deviation is incorporated into the system-level BO loss. Since different performance metrics span several orders of magnitude (e.g., preamplifier bandwidth on the order of $10^{[5,8]}$ versus comparator propagation delay on the order of $10^{[-10,-9]}$), larger values could overwhelm smaller ones. To normalize these differences, each loss is evaluatedon a logarithmic scale. For instance, if a bandwidth of $100M$ is desired but only $10M$ is achieved, the loss incurred is $log(100M) - log(10M) = 1$ instead of $100M - 10M = 90M$. Conveniently, all target values in this experiment are positive, allowing this approach; otherwise, a more advanced normalization method would be required, which is discussed in Section 4.5.

## 4.4 Experimental Results

The entire framework is implemented in Python 3.9, executed on a CentOS 7.9.2009, on an Intel Xeon Gold 5320 CPU with 96 cores. The experiments are performed using TSMC

28nm technology with Cadence's Spectre simulator, which restricts our transistor length, width, and number of fingers to [30nm, 1$\mu$m], [100nm, 3$\mu$m], and [1, 100] respectively. We use Bayesian optimization (BO) implemented by the Optuna [50] library, and limit our total simulation count to 2000. Subcircuit-level BO is parallelized at 20 concurrent threads.

Table 4.2 presents the results given two sets of performance specifications, compared against previous work. Table 4.3 presents the runtime breakdown in terms of real-time seconds and BO iterations. For reference, we also list the total number of parameters tuned in each subcircuit.

| **Resolution** | **12 bit** | **14 bit** |
|---|---|---|
| **Sampling rate** | **1MS/s** | **500KS/s** |
| **Unit Capacitor (2)** | 7 (34) | 11 (52) |
| **Bootstrap (22)** | 490 (31) | 1206 (98) |
| **Comparator (8)** | 981 (1315) | 20 (31) |
| **Pre-amplifier (30)** | 1661 (2147) | 2290 (2848) |
| **System-Level (4)** | 3764 (1) | 4111 (1) |

Table 4.3: Runtime Breakdown in seconds (BO iterations), number of parameter is listed next to subcircuit name.

Overall, the 14-bit 500 KS/s design achieves an SNDR of 80 dB with a power consumption $P$ of 153.6 µW, while the 12-bit 1 MS/s design achieves an SNDR of 70.3 dB with a power consumption of 72.81 µW. We calculate the Schreier and Walden performance metrics (FOM) as follows:

$$\mathrm{FOM}_S = \mathrm{SNDR} + 10\log\left(\frac{f_s}{2P}\right) \tag{4.10}$$

$$\mathrm{FOM}_W = \frac{P}{f_s \cdot 2^{\mathrm{ENOB}}} \tag{4.11}$$

where SNDR is the signal-to-noise-distortion ratio, $f_s$ is the sampling frequency, ENOB is the

effective number of bits for the SAR ADC, and $P$ is the power consumption. A comparison with previous work is presented in Table 4.2.

In the 14-bit 500 KS/s design, 62% of the power consumption is attributed to the SAR logic. This is due to the higher power consumption associated with the use of synchronous SAR logic. The power consumption might be further optimized if asynchronous SAR was used. In the 12-bit 1 MS/s design, the power consumption of comparator accounts for a larger proportion of the total power consumption, as the dynamic power of the comparator rises in high-speed designs.

In terms of optimization efficiency, as we increase the desired ADC resolution from 12 to 14 and reduce sampling frequency from 1M to 500k, we notice a change in runtime. The comparator requires fewer iterations to meet specifications, while the bootstrap circuit and the pre-amplifier take longer. This is because the higher resolution requirement sets harder specifications for the bootstrap and pre-amplifier circuits. Overall, the proposed framework arrives at a similar FoM and runtime to the previous work, showing the effectiveness of the automated method even without manual intervention.

## 4.5 Discussion and Future Work

In this work, we propose an autonomous SAR ADC sizing framework. We construct a dependency graph from predefined analytical equations, then use the topological order of the graph to serialize a system-level optimization loop. Within the loop, we partition system-level specifications to each subcircuit and optimize them individually. Finally, the flow verifies the design constraints and decides whether to continue the optimization loop. We verified the proposed framework on two different sets of performance specifications, and successfully produced functional SAR ADCs with comparable performance to previous autonomous sizing frameworks.

For future work, our method will be improved on three different fronts. Firstly, the acyclic

dependency assumption will be released, which allows us to support any set of analytical equations. Next, we will employ offline sizing methods, where the subcircuit-level BO are performed from pre-collected simulation data, so that we pay a one-time cost for all sizing, and therefore runtime no longer scales linearly with the number of system-level iterations. This should allow us to tackle less thoroughly collected equations and more aggressive empirical value ranges. Finally, we will expand our method to different types of circuits, as the current methodology is not designed specifically for SAR ADCs and may work for other hierarchical AMS designs as well.

# CHAPTER 5

# Conclusion

This dissertation presents a comprehensive, data-driven approach to automating analog/mixed-signal (AMS) circuit analysis and design. By addressing the longstanding challenges in AMS design—namely its reliance on manual effort, lack of explainability, and resistance to automation—we introduce a series of novel methodologies and systems that collectively move the field closer to fully automated design workflows.

We began by developing AMSnet, a multimodal database of schematics, netlists, and functional annotations. This dataset was made possible through a robust schematic parsing pipeline that combines traditional algorithms and machine learning techniques to reliably convert circuit images into structured netlists. The annotated data and knowledge graph organization of AMSnet provide rich, queryable insights that support downstream design tasks.

Building upon this foundation, we introduced AMSgen, an LLM-assisted design framework that autonomously translates performance specifications into complete circuit designs. AMSgen integrates natural language reasoning, knowledge graph retrieval, and Bayesian optimization in a feedback loop, enabling it to generate and refine topologies and device sizing without human intervention. This framework successfully produced designs for operational amplifiers and comparators, with layout-aware optimization through analog cell libraries (Acells).

To address the scalability limitations of simulation-heavy optimization, we proposed a hierarchical design flow for complex systems like SAR ADCs. By decomposing global speci-

fications into local targets and coordinating subcircuit optimization via dependency graphs, we significantly reduced computational overhead. This allowed for efficient end-to-end design of circuits with over 200 devices—without requiring full-circuit simulation in each iteration.

Together, these contributions demonstrate that combining LLMs, computer vision, structured datasets, and analytical reasoning can unlock practical solutions to AMS design automation. Our work bridges the gap between tacit expert knowledge and formalized computational models, offering new capabilities for both academia and industry.

Looking forward, several promising research directions emerge:

Testbench Generation: Automatically creating simulation setups for evaluating circuit performance would remove one of the last remaining manual steps in the design pipeline.

Self-supervised Annotation: Developing tools to automatically annotate functionality, symmetry constraints, and performance traits would further reduce the human cost of dataset expansion.

Improved Multimodal Understanding: Enhancing LLMs with native schematic interpretation—possibly through fine-tuning with AMSnet—could lead to models capable of directly parsing and generating circuit diagrams.

Design Debugging and Explainability: Integrating diagnostic tools that can explain design failures or suggest corrective actions would make the design process more transparent and trustworthy.

Broader Circuit Coverage: Extending the framework to handle more circuit classes such as filters, oscillators, and power management blocks would expand its applicability.

In conclusion, this dissertation sets the stage for the next generation of AMS design tools—ones that are data-driven, intelligent, and capable of emulating the decision-making processes of expert designers. By continuing to expand the dataset, refine the automation algorithms, and deepen the integration with domain knowledge, we envision a future where complex AMS design can be performed quickly, accurately, and autonomously.

# Appendix

# AMSgen: LLM Conversations

## Opamp Initial Design

**Prompt: Initial Design Strategy**

**Task:** Generate an operational amplifier. First analyze each constraint and provide a topology design following the given example.

**Requirements:** Using 28nm process with the following requirements:
DC gain > 80dB, GBW > 10MHz, phase margin > 60 degrees, CMRR > 80dB, PSRR > 80dB.
Follow the formats in the following example:

**Few-Shot ICL**

**Task:** Generate a comparator.

First analyze each constraint and provide a topology design.

**Requirements:** Using 28nm process with the following requirements:

Propagation delay < 2.5ns, power dissipation < 5mW, Offset Voltage < 50mV, Hysteresis Voltage < 50mV.

**Analyses:**

1. The requirement of propagation delay is smaller than 2.5ns, which means that the comparator must respond quickly to input changes, indicating high-speed circuitry and minimized capacitances. In 28nm process, this means that we can use the minimum sizes transistors to minimizes the MOS parasitic capacitances, thus benefiting the transient response.

2. Power Dissipation<5mW means that efficient biasing and low-power design techniques are necessary.

3. The offset voltage and the hysteresis voltage needs to be lower than 50mV, indicating that the comparator needs a pre-amplifier to obtain high DC gain in order to reduce the comparator input-referred offset, low kick-back noise, and high speed, while keeping low parasitic input capacitances.

**Circuit structure:**

**Input stage:**

-Differential pair input stage: Use a pair of MOSFETs as the differential pair input stage. Choose two PMOS with a suitable aspect ratio (W/L) to provide high gain.

-Current mirror load: Use current mirrors as loads to achieve controllable current and bias.

**Main comparator stage:**

-Latch as comparator: Using a regenerative latch as a comparator. Resistive comparing circuit for regenerative mode is used in series with NMOS transistor to get a low power consumption. Besides that, PMOS transistor for pre charging circuit is absent during reset mode and NMOS transistor for output pass transistor is nearly to Vdd/2 for the equalization of both voltage.

**Output buffer stage:**

-Buffer: The output buffer stage consists of a self-biased differential amplifier followed by an inverter which gives the digital output. Speed optimization:

-Fast transient response design: Use the minimized-size transistor to minimize the parasitic capacitance, but at the same time paying attention to the sensitivity of small-size transistors to process variations.

Figure 5.1: LLM prompt: from performance specification to initial design strategy

**LLM Response: Initial Design Strategy**

**Task: Generate an Operational Amplifier**
**Requirements:** Using 28nm process with the following specifications:

- DC gain > 80dB
- GBW > 10MHz
- Phase margin > 60 degrees
- CMRR > 80dB
- PSRR > 80dB

**Analyses:**

1. **DC Gain (> 80dB):** The requirement for a high DC gain indicates that the op-amp should have multiple stages to ensure a high level of amplification. A cascaded configuration can be used, typically starting with a differential input stage for high gain and good common-mode rejection.
2. **Gain-Bandwidth Product (GBW > 10MHz):** This necessitates a design that balances gain and frequency response. A compensation technique, like Miller compensation, might be employed in the gain stage to extend the bandwidth while maintaining stability.
3. **Phase Margin (> 60 degrees):** To achieve a stable operation with a phase margin greater than 60 degrees, careful design of the feedback network and internal compensation are required. This involves selecting appropriate capacitor values for the compensation network to ensure that the phase shift does not lead to instability.
4. **Common-Mode Rejection Ratio (CMRR > 80dB):** A high CMRR is crucial for reducing noise and interference that appears commonly on both inputs. This can be achieved by ensuring that the input stage has highly matched transistors and by using techniques such as common-mode feedback.
5. **Power Supply Rejection Ratio (PSRR > 80dB):** To achieve high PSRR, the power supply lines should be well decoupled. Additionally, using a differential architecture throughout can help minimize the effect of supply variations on the output.

**Circuit Structure:**
**Input Stage:**

- **Differential Pair:** Utilize a differential pair with NMOS transistors for the input stage to achieve high input impedance and initial gain. This stage will be the primary contributor to the CMRR.
- **Active Load:** Employ an active load (typically a current mirror) to convert the differential input signal into a single-ended output while increasing the gain.

**Gain Stage:**

- **Second Stage (Voltage Gain Stage):** A second amplification stage can further amplify the signal from the input stage. This stage typically uses a PMOS transistor for high gain and can be configured with Miller compensation to stabilize the gain and bandwidth.

**Compensation and Stability:**

- **Miller Compensation:** Implement Miller compensation across the second stage to control the frequency response and stabilize the op-amp by increasing the phase margin.

**Speed and Power Optimization:**

- **Optimized Transistor Sizing:** Use minimum size for the input differential pair to minimize parasitic capacitances, enhancing the speed. Carefully size the transistors in the input and gain stages to optimize the trade-off between speed, gain, and power consumption.
- **Biasing:** Efficiently bias all stages to ensure that the op-amp operates within the power constraints while maintaining linearity and minimizing power dissipation.

This approach will meet the stringent requirements set out for the operational amplifier in a 28nm CMOS process, achieving a balance between performance parameters and practical design constraints.

Figure 5.2: LLM response: from performance specification to initial design strategy

**Prompt: Design Strategy to Relation Pairs**

**Task:** Extract the relationships from the following text into relation pairs for knowledge graph query. Each pair should be in the form of: <RELATION, OBJECT>.

**Note:** RELATION should be performance metrics or building blocks, i.e. gain, power consumption, input, load, etc.
**Note:** OBJECT should be qualitative description for the performance or building block selection, i.e. high, fast, current mirror, latch, etc.
**Example pairs:**
<gain, high>
<offset voltage, low>
<power consumption, low>
<input, differential pair>

**Circuit Structure:**
**Input Stage:**
- **Differential Pair:** Utilize a differential pair with NMOS transistors for the input stage to achieve high input impedance and initial gain. This stage will be the primary contributor to the CMRR.
- **Active Load:** Employ an active load (typically a current mirror) to convert the differential input signal into a single-ended output while increasing the gain.

**Gain Stage:**
- **Second Stage (Voltage Gain Stage):** A second amplification stage can further amplify the signal from the input stage. This stage typically uses a PMOS transistor for high gain and can be configured with Miller compensation to stabilize the gain and bandwidth.

**Compensation and Stability:**
- **Miller Compensation:** Implement Miller compensation across the second stage to control the frequency response and stabilize the op-amp by increasing the phase margin.

**Speed and Power Optimization:**
- **Optimized Transistor Sizing:** Use minimum size for the input differential pair to minimize parasitic capacitances, enhancing the speed. Carefully size the transistors in the input and gain stages to optimize the trade-off between speed, gain, and power consumption.
- **Biasing:** Efficiently bias all stages to ensure that the op-amp operates within the power constraints while maintaining linearity and minimizing power dissipation.

Figure 5.3: LLM prompt: from initial design strategy to relation pairs

**LLM Response: Extract Relation Pairs**

Based on the given text, the extracted relation pairs for constructing a knowledge graph are as follows:

1. <input impedance, high>
2. <initial gain, high>
3. <gain, increased>
4. <gain, high>
5. <gain stabilization, Miller compensation>
6. <bandwidth stabilization, Miller compensation>
7. <speed, enhanced>
8. <gain and power optimization, transistor sizing>
9. <linearity, maintained>
10. <power dissipation, minimized>
11. <input, differential pair>
12. <load, active (current mirror)>

Figure 5.4: LLM response: from initial design strategy to relation pairs

## Opamp Regenerated Design

**Prompt: Revised Design Strategy**

**Task:** Generate an operational amplifier. First analyze each constraint and provide a topology design following the given example.
**Requirements:** Using 28nm process with the following requirements:
DC gain > 80dB, GBW > 10MHz, phase margin > 60 degrees, CMRR > 80dB, PSRR > 80dB.
Follow the formats in the following example:

**Few-Shot ICL**

**Task: Generate an Operational Amplifier**
**Requirements:** Using 28nm process with the following specifications:

- DC gain > **66.21** dB
- GBW > **319** MHz
- Phase margin > **63.63** degrees
- CMRR > **54.2** dB
- PSRR > **69.82** dB

**Analyses:**

1. **DC Gain (> 66.21 dB):** The requirement for a high DC gain indicates that the op-amp should have multiple stages to ensure a high level of amplification. A cascaded configuration can be used, typically starting with a differential input stage for high gain and good common-mode rejection.
2. **Gain-Bandwidth Product (GBW > 319 MHz):** This necessitates a design that balances gain and frequency response. A compensation technique, like Miller compensation, might be employed in the gain stage to extend the bandwidth while maintaining stability.
3. **Phase Margin (> 63.63 degrees):** To achieve a stable operation with a phase margin greater than 60 degrees, careful design of the feedback network and internal compensation is required. This involves selecting appropriate capacitor values for the compensation network to ensure that the phase shift does not lead to instability.
4. **Common-Mode Rejection Ratio (CMRR > 54.2 dB):** A high CMRR is crucial for reducing noise and interference that appears commonly on both inputs. This can be achieved by ensuring that the input stage has highly matched transistors and by using techniques such as common-mode feedback.
5. **Power Supply Rejection Ratio (PSRR > 69.82 dB):** To achieve high PSRR, the power supply lines should be well decoupled. Additionally, using a differential architecture throughout can help minimize the effect of supply variations on the output.

**Circuit Structure:**
**Input Stage:**

- **Differential Pair:** Utilize a differential pair with NMOS transistors for the input stage to achieve high input impedance and initial gain. This stage will be the primary contributor to the CMRR.
- **Active Load:** Employ an active load (typically a current mirror) to convert the differential input signal into a single-ended output while increasing the gain.

**Gain Stage:**

- **Second Stage (Voltage Gain Stage):** A second amplification stage can further amplify the signal from the input stage. This stage typically uses a PMOS transistor for high gain and can be configured with Miller compensation to stabilize the gain and bandwidth.

**Compensation and Stability:**

- **Miller Compensation:** Implement Miller compensation across the second stage to control the frequency response and stabilize the op-amp by increasing the phase margin.

**Speed and Power Optimization:**

- **Optimized Transistor Sizing:** Use minimum size for the input differential pair to minimize parasitic capacitances, enhancing the speed. Carefully size the transistors in the input and gain stages to optimize the trade-off between speed, gain, and power consumption.
- **Biasing:** Efficiently bias all stages to ensure that the op-amp operates within the power constraints while maintaining linearity and minimizing power dissipation.

Figure 5.5: LLM prompt: from performance specification to revised design strategy

**LLM Response: Revised Design Strategy**

**Task: Generate an Operational Amplifier**
**Requirements:** Using 28nm process with the following specifications:

- DC gain > 80dB
- GBW > 10MHz
- Phase margin > 60 degrees
- CMRR > 80dB
- PSRR > 80dB

**Analyses:**

1. **DC Gain (> 80 dB):** A high DC gain suggests the necessity for a multi-stage op-amp. A common approach is to employ a two-stage architecture, beginning with a differential input stage, which provides substantial initial gain and excellent common-mode rejection capabilities.
2. **Gain-Bandwidth Product (GBW > 10 MHz):** To meet this requirement while maintaining a high DC gain, it is essential to use frequency compensation techniques such as Miller compensation in the second gain stage. This helps in achieving a wide bandwidth without sacrificing stability.
3. **Phase Margin (> 60 degrees):** Ensuring a phase margin greater than 60 degrees is crucial for the stability of the op-amp. This can be achieved by careful design of the compensation network, possibly incorporating a Miller capacitor that connects the output of the second stage back to an intermediate stage.
4. **Common-Mode Rejection Ratio (CMRR > 80 dB):** High CMRR is essential for applications requiring accuracy and noise immunity. This can be achieved by utilizing well-matched transistor pairs in the differential input stage and employing techniques like common-mode feedback to enhance symmetry.
5. **Power Supply Rejection Ratio (PSRR > 80 dB):** High PSRR is achieved by minimizing the impact of power supply fluctuations on the amplifier's output. This involves the use of regulated supply voltages and integrating a rejection mechanism such as differential stages throughout the design to reject common-mode signals from the power supply.

**Circuit Structure:**
**Input Stage:**

- **Differential Pair:** Use a differential pair with NMOS transistors for the input to achieve high input impedance and significant initial gain. This configuration is the primary contributor to both the DC gain and CMRR.
- **Active Load:** Implement an active load, likely a cascode current mirror, to convert the differential voltage into a larger single-ended output, significantly increasing the gain and improving linearity.

**Gain Stage:**

- **Second Stage (Voltage Gain Stage):** A second amplification stage will further amplify the signal from the input stage. This stage typically uses a PMOS transistor for improved gain characteristics and is where Miller compensation can be applied.

**Compensation and Stability:**

- **Miller Compensation:** Implement Miller compensation across the second stage. This method involves connecting a compensation capacitor between the output and an intermediate point (usually the gate of a transistor in the first stage), effectively controlling the frequency response and stabilizing the op-amp by increasing the phase margin.

**Speed and Power Optimization:**

- **Optimized Transistor Sizing:** Use minimum size transistors in the input differential pair to reduce parasitic capacitances, thereby enhancing speed. The sizing of transistors in all stages must balance speed, gain, and power consumption.
- **Biasing:** Ensure efficient biasing of all stages to maintain op-amp operation within power constraints, optimizing performance across varying operating conditions and minimizing power dissipation.

This design strategy should enable the realization of an operational amplifier meeting the stringent requirements set for a 28nm CMOS process, balancing performance parameters and practical design constraints effectively.

Figure 5.6: LLM response: from performance specification to revised design strategy

**Prompt: Design Strategy to Relation Pairs**

**Task:** Extract the relationships from the following text into relation pairs for knowledge graph query. Each pair should be in the form of: <RELATION, OBJECT>.

**Note:** RELATION should be performance metrics or building blocks, i.e. gain, power consumption, input, load, etc.
**Note:** OBJECT should be qualitative description for the performance or building block selection, i.e. high, fast, current mirror, latch, etc.
**Example pairs:**
<gain, high>
<offset voltage, low>
<power consumption, low>
<input, differential pair>

### Circuit Structure:

**Input Stage:**

- **Differential Pair:** Use a differential pair with NMOS transistors for the input to achieve high input impedance and significant initial gain. This configuration is the primary contributor to both the DC gain and CMRR.
- **Active Load:** Implement an active load, likely a cascode current mirror, to convert the differential voltage into a larger single-ended output, significantly increasing the gain and improving linearity.

**Gain Stage:**

- **Second Stage (Voltage Gain Stage):** A second amplification stage will further amplify the signal from the input stage. This stage typically uses a PMOS transistor for improved gain characteristics and is where Miller compensation can be applied.

**Compensation and Stability:**

- **Miller Compensation:** Implement Miller compensation across the second stage. This method involves connecting a compensation capacitor between the output and an intermediate point (usually the gate of a transistor in the first stage), effectively controlling the frequency response and stabilizing the op-amp by increasing the phase margin.

**Speed and Power Optimization:**

- **Optimized Transistor Sizing:** Use minimum size transistors in the input differential pair to reduce parasitic capacitances, thereby enhancing speed. The sizing of transistors in all stages must balance speed, gain, and power consumption.
- **Biasing:** Ensure efficient biasing of all stages to maintain op-amp operation within power constraints, optimizing performance across varying operating conditions and minimizing power dissipation.

This design strategy should enable the realization of an operational amplifier meeting the stringent requirements set for a 28nm CMOS process, balancing performance parameters and practical design constraints effectively.

Figure 5.7: LLM prompt: from revised design strategy to relation pairs

**LLM Response: Extract Relation Pairs**

Based on the provided text, here are the relation pairs extracted for use in a knowledge graph query:

1. <input impedance, high>
2. <gain, high>
3. <DC gain, primary contributor>
4. <CMRR, primary contributor>
5. <gain, increased>
6. <linearity, improved>
7. <gain, improved>
8. <Miller compensation, applied>
9. <frequency response, controlled>
10. <stability, increased>
11. <phase margin, increased>
12. <speed, enhanced>
13. <gain, balanced>
14. <power consumption, balanced>
15. <power dissipation, minimized>
16. <input, differential pair>
17. <active load, cascode current mirror>
18. <transistor sizing, minimum>

Figure 5.8: LLM response: from revised design strategy to relation pairs

## Comparator Design

**Prompt: Design Strategy**

**Task:** Generate a comparator. First analyze each constraint and provide a topology design following the given example..

**Requirements：** Using 28nm process with the following requirements:
propagation delay below 1ns, offset voltage below 100uV, sampling frequency 1GHz, power dissipation below 100uW.
Follow the formats in the following example:

**Few-Shot ICL**

**Task:** Generate an Operational Amplifier Analyze each constraint and provide a topology design following the given example.

**Requirements:**

Using a 28nm process with the following specifications:

DC Gain > 80dB, GBW > 10MHz, Phase Margin > 60, degrees CMRR > 80dB, PSRR > 80dB

**Analyses:**

**DC Gain > 80dB:**

This high gain requirement suggests the need for a robust first stage with a good intrinsic gain. Using a differential pair with active loads can help achieve high initial gain.

**GBW > 10MHz:**

A high gain-bandwidth product indicates the necessity of a carefully designed compensation strategy. This typically involves placing a compensation capacitor between the output of the first stage and the output of the second stage to ensure stability and adequate bandwidth.

**Phase Margin > 60 Degrees:**

Maintaining a high phase margin requires careful feedback and compensation network design to ensure stability, especially at high frequencies. This often involves selecting appropriate values for the compensation capacitors and possibly incorporating additional poles or zeros strategically.

**CMRR > 80dB and PSRR > 80dB:**

Both requirements point towards a need for excellent matching and symmetry in the differential input stage and minimizing supply and common-mode noise influence. Techniques like using matched transistor pairs and common centroid layout can help improve CMRR and PSRR.

**Circuit Structure:**

**Input Stage: Differential Pair with Active Load:**

Utilize a pair of NMOS transistors as the differential input pair. Choose an appropriate aspect ratio (W/L) to maximize the transconductance and, consequently, the gain. Employ a PMOS current mirror as the active load to enhance the output impedance and thus the gain of this stage.

**Gain Stage (Second Stage): Common-Source Stage with Active Load:**

The second stage can be a common-source amplifier driven by the output of the first stage. This stage further amplifies the voltage and drives the output load. Another PMOS can be used as the active load to further improve the gain. Compensation: Place a compensation capacitor and a resistor between the output of the first stage and the output of the second stage to stabilize the two-stage amplifier configuration and set the dominant pole for desired phase margin and bandwidth.

**Speed and Stability Optimization:**

Minimize Parasitic Effects: use minimum length for transistors where possible to reduce parasitic capacitances, enhancing speed. Enhanced Matching: Implement layout techniques such as common centroid and interdigitated fingers for critical matched transistors to improve CMRR and PSRR by reducing mismatches. This topology should meet the stringent requirements set for the operational amplifier while utilizing the advanced capabilities of the 28nm process technology.

Figure 5.9: LLM prompt: from performance specification to design strategy

**LLM Response: Design Strategy**

**Task**: Generate a Comparator Analyze each constraint and provide a topology design following the given example.

**Requirements**: Using a 28nm process with the following specifications:

Propagation delay < 1ns, Offset voltage < 100uV, Sampling frequency = 1GHz, Power dissipation < 100uW

**Analyses**:

**Propagation Delay < 1ns:**

This extremely low delay requirement indicates the need for a fast response time, which can be achieved by a simple topology with minimal stages and using devices with high transconductance. Minimizing the capacitive load on the comparator output is also crucial.

**Offset Voltage < 100uV:**

Achieving such a low offset voltage requires excellent matching of the input transistors. Techniques like common centroid layout and dummy transistors can be used to enhance matching. Using a preamplifier stage with auto-zeroing techniques may also be beneficial to mitigate offset.

**Sampling Frequency 1GHz:**

High-frequency operation demands the comparator to be optimized for speed. This involves reducing the total parasitic capacitance and ensuring that the current drive capability of the output stage is sufficient to handle the load quickly.

**Power Dissipation < 100uW:**

Low power consumption can be maintained by optimizing the biasing currents and utilizing minimum size devices where possible, balancing speed requirements.

**Circuit Structure:**

**Input Stage: Differential Pair with Cross-Coupled Load:**

Use a pair of NMOS transistors for the differential input. These should have minimum channel length to maximize speed and carefully sized to ensure good matching. Implement a cross-coupled pair of PMOS transistors (M3 and M4) as the active load. This configuration helps in rapidly pulling the output to the supply rails, enhancing the switching speed.

**Latch Stage: Positive Feedback Latch:**

A latch stage can be incorporated to quickly amplify the differential signal from the input stage and drive the output decisively to the digital logic levels. This stage uses additional NMOS and PMOS transistors to form a regenerative loop, ensuring fast decision times.

**Output Buffer:**

An output buffer may be necessary to drive the external load, especially at high frequencies. This stage should be designed to minimize power consumption while still being capable of fast switching.

**Speed and Stability Optimization**:

Minimize Parasitic Effects: Employ minimum length for all transistors and minimize routing capacitances to enhance the speed of the comparator.

Enhanced Matching: Use layout techniques such as common centroid and interdigitated fingers for the input matched transistors to reduce mismatches which contribute to offset voltage.

Power Optimization: Choose a bias current that balances speed and power requirements, avoiding excessive current that leads to high power dissipation.

Figure 5.10: LLM response: from performance specification to design strategy

**Prompt: Design Strategy to Relation Pairs**

**Task:** Extract the relationships from the following text into relation pairs for knowledge graph query. Each pair should be in the form of: <RELATION, OBJECT>.

**Note:** RELATION should be performance metrics or building blocks, i.e. gain, power consumption, input, load, etc.
**Note:** OBJECT should be qualitative description for the performance or building block selection, i.e. high, fast, current mirror, latch, etc.
**Example pairs:**
<gain, high>
<offset voltage, low>
<power consumption, low>
<input, differential pair>

**Circuit Structure:**
**Input Stage: Differential Pair with Cross-Coupled Load:**
Use a pair of NMOS transistors for the differential input. These should have minimum channel length to maximize speed and carefully sized to ensure good matching.
Implement a cross-coupled pair of PMOS transistors as the active load. This configuration helps in rapidly pulling the output to the supply rails, enhancing the switching speed.
**Latch Stage: Positive Feedback Latch:**
A latch stage can be incorporated to quickly amplify the differential signal from the input stage and drive the output decisively to the digital logic levels. This stage uses additional NMOS and PMOS transistors to form a regenerative loop, ensuring fast decision times.
**Output Buffer:**
An output buffer may be necessary to drive the external load, especially at high frequencies. This stage should be designed to minimize power consumption while still being capable of fast switching.
**Speed and Stability Optimization**:
Minimize Parasitic Effects: Employ minimum length for all transistors and minimize routing capacitances to enhance the speed of the comparator.
Enhanced Matching: Use layout techniques such as common centroid and interdigitated fingers for the input matched transistors to reduce mismatches which contribute to offset voltage.
**Power Optimization:**
Choose a bias current that balances speed and power requirements, avoiding excessive current that leads to high power dissipation.

Figure 5.11: LLM prompt: from design strategy to relation pairs

**LLM Response: Extract Relation Pairs**

Based on the description provided, here are the relation pairs extracted for constructing a knowledge graph query:

1. <input, differential pair>
2. <speed, maximized>
3. <matching, good>
4. <output, fast>
5. <power consumption, minimized>
6. <switching speed, enhanced>
7. <decision time, fast>
8. <load, external>
9. <switching, fast>
10. <parasitic effects, minimized>
11. <matching, enhanced>
12. <offset voltage, reduced>
13. <speed, balanced>
14. <power dissipation, high>

Figure 5.12: LLM response: from design strategy to relation pairs

**Prompt: Topology Selection**

**Task**: Our database returns 2 matching topologies:

- Strong-arm latch
- Double tail latch

Select a single topology between the above.

**LLM Response: Topology Selection**

**Answer**: Given the requirements and the matching topologies available from your database, I would recommend selecting the **Strong-arm latch** topology for your comparator design.

Here's why:

**Speed and Power Efficiency:**

The Strong-arm latch is known for its excellent speed characteristics and low power consumption. This fits well with your requirement of having a propagation delay below 1ns and power dissipation below 100uW.

**Offset Voltage Management:**

The Strong-arm topology, due to its symmetric design, can be optimized for low offset voltage through careful layout and transistor matching, addressing the requirement of keeping offset voltage below 100uV.

**High Sampling Frequency Compatibility:**

This topology is effective in handling high sampling frequencies, in your case, 1GHz, due to its inherent quick reset and set-up characteristics, making it suitable for high-speed applications.

**Simplicity and Robustness**:

Strong-arm latches are relatively simple and robust, providing a good balance between performance and design complexity, which can be advantageous in a 28nm process where layout density and variability might be concerns.

This topology should serve well in meeting the stringent performance criteria set by your specifications while utilizing the advanced capabilities of the 28nm process technology.

Figure 5.13: LLM prompt and response: tie breaker between valid topologies, continuing on the context of Fig. 5.10

# REFERENCES


[1] A. Mehradfar, X. Zhao, Y. Niu, S. Babakniya, M. Alesheikh, H. Aghasi, and S. Avestimehr, "AICircuit: A Multi-Level Dataset and Benchmark for AI-Driven Analog Integrated Circuit Design," *arXiv preprint arXiv:2407.18272*, 2024.

[2] Infineon, "adsbenchmark." `https://rb.gy/1m2263`.

[3] ALIGN Project, "ALIGN Public Repository." `https://github.com/ALIGN-analoglayout/ALIGN-public`.

[4] H. Chen, K. Zhu, M. Liu, X. Tang, N. Sun, and D. Z. Pan, "Universal Symmetry Constraint Extraction for Analog and Mixed-Signal Circuits with Graph Neural Networks," in *P2021 58th ACM/IEEE Design Automation Conference (DAC)*, pp. 1243–1248, IEEE, 2021.

[5] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 779–788, 2016.

[6] J. P. Lewis *et al.*, "Fast Template Matching," in *Vision interface*, vol. 95, pp. 15–19, Quebec City, QC, Canada, 1995.

[7] B. Razavi, "Design of Analog CMOS Integrated Circuits," 2000.

[8] D. Bolya, C. Zhou, F. Xiao, and Y. J. Lee, "YOLACT: Real-Time Instance Segmentation," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 9157–9166, 2019.

[9] G. Jocher and J. Qiu, "Ultralytics YOLO11," 2024.

[10] A. M. Hafiz and G. M. Bhat, "A Survey on Instance Segmentation: State of the Art," *International Journal of Multimedia Information Retrieval*, vol. 9, no. 3, pp. 171–189, 2020.

[11] O. Ronneberger, P. Fischer, and T. Brox, "U-net: Convolutional Networks for Biomedical Image Segmentation," in *The 18th International Conference of Medical Image Computing and Computer-assisted Intervention (MICCAI)*, pp. 234–241, Springer, 2015.

[12] P. Michel, U. Lauther, and P. Duzy, *The Synthesis Approach to Digital System Design*, vol. 170. Springer Science & Business Media, 1992.

[13] R. A. Rutenbar, "Analog Circuit and Layout Synthesis Revisited," in *Proceedings of the 2015 Symposium on International Symposium on Physical Design* (A. Davoodi and E. F. Y. Young, eds.), p. 83, ACM, 2015.

[14] R. Zhong, X. Du, S. Kai, Z. Tang, S. Xu, H.-L. Zhen, J. Hao, Q. Xu, M. Yuan, and J. Yan, "LLM4EDA: Emerging Progress in Large Language Models for Electronic Design Automation," *arXiv preprint arXiv:2401.12224*, 2023.

[15] Y. Lai, S. Lee, G. Chen, S. Poddar, M. Hu, D. Z. Pan, and P. Luo, "AnalogCoder: Analog Circuit Design via Training-Free Code Generation," *arXiv preprint arXiv:2405.14918*, 2024.

[16] Y. Lu, S. Liu, Q. Zhang, and Z. Xie, "RTLLM: An Open-Source Benchmark for Design RTL Generation with Large Language Model," in *2024 29th Asia and South Pacific Design Automation Conference (ASP-DAC)*, pp. 722–727, IEEE, 2024.

[17] S. Thakur, B. Ahmad, Z. Fan, H. Pearce, B. Tan, R. Karri, B. Dolan-Gavitt, and S. Garg, "Benchmarking Large Language Models for Automated Verilog RTL Code Generation," in *2023 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, pp. 1–6, IEEE, 2023.

[18] Z. Chen, J. Huang, Y. Liu, F. Yang, L. Shang, D. Zhou, and X. Zeng, "Artisan: Automated Operational Amplifier Design via Domain-specific Large Language Model," in *2024 61th ACM/IEEE Design Automation Conference (DAC)*, pp. 1–6, 2024.

[19] Y. Yin, Y. Wang, B. Xu, and P. Li, "ADO-LLM: Analog Design Bayesian Optimization with In-Context Learning of Large Language Models," *arXiv preprint arXiv:2406.18770*, 2024.

[20] C. Liu, Y. Liu, Y. Du, and L. Du, "LADAC: Large Language Model-driven Auto-Designer for Analog Circuits," *Authorea Preprints*, 2024.

[21] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, D. Zhou, *et al.*, "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," *Advances in Neural Information Processing Systems*, vol. 35, pp. 24824–24837, 2022.

[22] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "React: Synergizing Reasoning and Acting in Language Models," *arXiv preprint arXiv:2210.03629*, 2022.

[23] Y. Hu, C.-H. Lee, T. Xie, T. Yu, N. A. Smith, and M. Ostendorf, "In-Context Learning for Few-Shot Dialogue State Tracking," *arXiv preprint arXiv:2203.08568*, 2022.

[24] J. Liu, D. Shen, Y. Zhang, B. Dolan, L. Carin, and W. Chen, "What Makes Good In-Context Examples for GPT-3?," *arXiv preprint arXiv:2101.06804*, 2021.

[25] C. Mattiussi and D. Floreano, "Analog Genetic Encoding for the Evolution of Circuits and Networks," *IEEE Transactions on evolutionary computation*, vol. 11, no. 5, pp. 596–607, 2007.

[26] T. McConaghy, P. Palmers, M. Steyaert, and G. G. Gielen, "Trustworthy Genetic Programming-Based Synthesis of Analog Circuit Topologies Using Hierarchical Domain-Specific Building Blocks," *IEEE Transactions on Evolutionary Computation*, vol. 15, no. 4, pp. 557–570, 2011.

[27] X. Shi, L. L. Minku, and X. Yao, "Tree-Based Genetic Programming for Evolutionary Analog Circuit with Approximate Shapley Value," in *International Conference on Innovative Techniques and Applications of Artificial Intelligence*, pp. 253–267, Springer, 2024.

[28] I. Abel and H. Graeb, "FUBOCO: Structure synthesis of basic op-amps by FUnctional BlOck composition," *ACM Transactions on Design Automation of Electronic Systems (TODAES)*, vol. 27, no. 6, pp. 1–27, 2022.

[29] Z. Zhao and L. Zhang, "Analog Integrated Circuit Topology Synthesis With Deep Reinforcement Learning," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 41, no. 12, pp. 5138–5151, 2022.

[30] Z. Dong, W. Cao, M. Zhang, D. Tao, Y. Chen, and X. Zhang, "CktGNN: Circuit Graph Neural Network for Electronic Design Automation," *arXiv preprint arXiv:2308.16406*, 2023.

[31] J. Lu, L. Lei, J. Huang, F. Yang, L. Shang, and X. Zeng, "Automatic op-amp generation from specification to layout," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2023.

[32] S. Thakur, B. Ahmad, H. Pearce, B. Tan, B. Dolan-Gavitt, R. Karri, and S. Garg, "VeriGen: A Large Language Model for Verilog Code Generation," *ACM Trans. Des. Autom. Electron. Syst.*, vol. 29, Apr. 2024.

[33] C.-C. Chang, Y. Shen, S. Fan, J. Li, S. Zhang, N. Cao, Y. Chen, and X. Zhang, "LaMAGIC: Language-Model-based Topology Generation for Analog Integrated Circuits," *arXiv preprint arXiv:2407.18269*, 2024.

[34] J. Shen, Z. Chen, J. Zhuang, J. Huang, F. Yang, L. Shang, Z. Bi, C. Yan, D. Zhou, and X. Zeng, "Atelier: An Automated Analog Circuit Design Framework via Multiple Large Language Model-based Agents," *Authorea Preprints*, 2024.

[35] N. Horta, "Analogue and Mixed-Signal Systems Topologies Exploration Using Symbolic Methods," *Analog Integrated Circuits and Signal Processing*, vol. 31, pp. 161–176, 2002.

[36] R. Phelps, M. Krasnicki, R. A. Rutenbar, L. R. Carley, and J. R. Hellums, "Anaconda: Simulation-Based Synthesis of Analog Circuits Via Stochastic Pattern Search," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 19, no. 6, pp. 703–717, 2000.

[37] R. A. Vural and T. Yildirim, "Analog Circuit Sizing via Swarm Intelligence," *AEU-International Journal of Electronics and Communications*, vol. 66, no. 9, pp. 732–740, 2012.

[38] B. Liu, Y. Wang, Z. Yu, L. Liu, M. Li, Z. Wang, J. Lu, and F. V. Fernández, "Analog Circuit Optimization System Based on Hybrid Evolutionary Algorithms," *Integration*, vol. 42, no. 2, pp. 137–148, 2009.

[39] E. Sağlican and E. Afacan, "MOEA/D vs. NSGA-II: A Comprehensive Comparison for Multi/Many Objective Analog/RF Circuit Optimization through a Generic Benchmark," *ACM Transactions on Design Automation of Electronic Systems*, vol. 29, no. 1, pp. 1–23, 2023.

[40] L.-Y. Song, C.-Y. Chou, T.-C. Kuo, C.-N. Liu, and J.-D. Huang, "Machine Learning Assisted Circuit Sizing Approach for Low-Voltage Analog Circuits with Efficient Variation-Aware Optimization," *ACM Trans. Des. Autom. Electron. Syst.*, vol. 28, Dec. 2022.

[41] T. Liao and L. Zhang, "Efficient Parasitic-aware gm/ID-based Hybrid Sizing Methodology for Analog and RF Integrated Circuits," *ACM Trans. Des. Autom. Electron. Syst.*, vol. 26, Oct. 2020.

[42] E. Saundefinedlican and E. Afacan, "MOEA/D vs. NSGA-II: A Comprehensive Comparison for Multi/Many Objective Analog/RF Circuit Optimization through a Generic Benchmark," *ACM Trans. Des. Autom. Electron. Syst.*, vol. 29, Nov. 2023.

[43] W. W. Xing, W. Fan, Z. Liu, Y. Yao, and Y. Hu, "KATO: Knowledge Alignment and Transfer for Transistor Sizing of Different Design and Technology," *arXiv preprint arXiv:2404.14433*, 2024.

[44] B. He, S. Zhang, F. Yang, C. Yan, D. Zhou, and X. Zeng, "An Efficient Bayesian Optimization Approach for Analog Circuit Synthesis via Sparse Gaussian Process Modeling," in *2020 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, pp. 67–72, IEEE, 2020.

[45] T. Gu, J. Wang, Z. Bi, C. Yan, F. Yang, Y. Qin, T. Cui, and X. Zeng, "tSS-BO: Scalable Bayesian Optimization for Analog Circuit Sizing via Truncated Subspace Sampling," in *2024 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, pp. 1–6, IEEE, 2024.

[46] A. Zhao, T. Gu, Z. Bi, F. Yang, C. Yan, X. Zeng, Z. Lin, W. Hu, and D. Zhou, "D3PBO: Dynamic Domain Decomposition-based Parallel Bayesian Optimization for Large-scale Analog Circuit Sizing," *ACM Trans. Des. Autom. Electron. Syst.*, vol. 29, Mar. 2024.

[47] K. S. NS and P. Li, "Pareto Optimization of Analog Circuits Using Reinforcement Learning," *ACM Trans. Des. Autom. Electron. Syst.*, vol. 29, Feb. 2024.

[48] Y. Uhlmann, T. Moldenhauer, and J. Scheible, "Differentiable Neural Network Surrogate Models for gm/ID-based Analog IC Sizing Optimization," in *2023 ACM/IEEE 5th Workshop on Machine Learning for CAD (MLCAD)*, pp. 1–6, 2023.

[49] H. Wang, K. Wang, J. Yang, L. Shen, N. Sun, H.-S. Lee, and S. Han, "GCN-RL Circuit Designer: Transferable Transistor Sizing with Graph Neural Networks and Reinforcement Learning," in *2020 57th ACM/IEEE Design Automation Conference (DAC)*, pp. 1–6, IEEE, 2020.

[50] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A Next-Generation Hyperparameter Optimization Framework," in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pp. 2623–2631, 2019.

[51] T. Zhou, C. Chang, L. Huang, J. Gu, Z. Ji, X. Liu, H. Liang, Z. Chen, T.-J. Lin, S. Wang, *et al.*, "Abuttable analog cell library and automatic ams layout," in *Proceedings of the 2025 International Symposium on Physical Design*, pp. 191–199, 2025.

[52] A. F. Budak, P. Bhansali, B. Liu, N. Sun, D. Z. Pan, and C. V. Kashyap, "Dnn-opt: An rl inspired optimization for analog circuit sizing using deep neural networks," in *2021 58th ACM/IEEE Design Automation Conference (DAC)*, pp. 1219–1224, IEEE, 2021.

[53] X. Tang, J. Liu, Y. Shen, S. Li, L. Shen, A. Sanyal, K. Ragab, and N. Sun, "Low-power sar adc design: Overview and survey of state-of-the-art techniques," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 69, no. 6, pp. 2249–2262, 2022.

[54] B. Murmann, "The successive approximation register adc: a versatile building block for ultra-low-power to ultra-high-speed applications," *IEEE Communications Magazine*, vol. 54, no. 4, pp. 78–83, 2016.

[55] W. Li, Z. Bi, and X. Zeng, "High-dimensional analog circuit sizing via bayesian optimization in the variational autoencoder enhanced latent space," in *2024 2nd International Symposium of Electronics Design Automation (ISEDA)*, pp. 193–197, IEEE, 2024.

[56] R. Mina, C. Jabbour, and G. E. Sakr, "A review of machine learning techniques in analog integrated circuit design automation," *Electronics*, vol. 11, no. 3, p. 435, 2022.

[57] C.-P. Huang, J.-M. Lin, Y.-T. Shyu, and S.-J. Chang, "A systematic design methodology of asynchronous sar adcs," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 24, no. 5, pp. 1835–1848, 2015.

[58] M. Ding, P. Harpe, G. Chen, B. Busze, Y.-H. Liu, C. Bachmann, K. Philips, and A. van Roermund, "A hybrid design automation tool for sar adcs in iot," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 26, no. 12, pp. 2853–2862, 2018.

[59] M. Liu, X. Tang, K. Zhu, H. Chen, N. Sun, and D. Z. Pan, "Opensar: An open source automated end-to-end sar adc compiler," in *2021 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*, pp. 1–9, IEEE, 2021.

[60] M. Hassanpourghadi, R. A. Rasul, and M. S.-W. Chen, "A module-linking graph assisted hybrid optimization framework for custom analog and mixed-signal circuit parameter synthesis," *ACM Transactions on Design Automation of Electronic Systems (TODAES)*, vol. 26, no. 5, pp. 1–22, 2021.